数据结构与算法：第九章 图

哈尔滨工业大学（深圳）期末复习文档

使用说明

每个知识模块均包含：

1. 【考点体系梳理】：梳理知识逻辑，明确概念之间的关系。
2. 【核心概念与公式】：集中列出定义、性质、公式、适用条件和复杂度。
3. 【典型题型精析】：按照“题干呈现 $\rightarrow$ 解题思路分析 $\rightarrow$ 详细解答步骤 $\rightarrow$ 易错点与总结”展开。
4. 【自查与巩固建议】：通过提问、填空和练习方向完成自我检测。

文中统一使用：

- V：顶点数；
- E：边数或弧数；
- INF：两点之间当前不可达；
- 若题目未规定邻接点访问顺序，默认按顶点编号或字母升序访问。

---

1. 图的基本概念与存储结构

1.1 【考点体系梳理】

本节知识逻辑如下：

图的定义
├─ 顶点与边
├─ 有向图与无向图
├─ 带权图与无权图
├─ 路径、回路与连通性
└─ 图的存储
  ├─ 邻接矩阵
  └─ 邻接表

考试通常不会只问某个定义，而是把多个环节串联起来：

给出图形
→ 写邻接矩阵或邻接表
→ 计算度数
→ 进行 DFS/BFS
→ 分析时间、空间复杂度

需要特别明确以下关系：

- 无向图中的一条边同时关联两个顶点，因此所有顶点度数之和为 $2E$。
- 有向图中的一条弧对弧尾贡献一个出度，对弧头贡献一个入度。
- 存储结构会直接影响邻接点查询效率，也会影响 DFS、BFS 的实现方式。
- 邻接点排列顺序不是唯一的，因此基于邻接表得到的遍历序列也可能不唯一。

1.2 【核心概念与公式】

1.2.1 图的定义

图可以表示为：

$$G = (V, E)$$

其中，V 是非空顶点集合，E 是边或弧的集合。

1.2.2 度数公式

无向图：

$$\sum_{v \in V} \deg(v) = 2E$$

有向图：

$$\sum_{v \in V} \mathrm{indegree}(v) = \sum_{v \in V} \mathrm{outdegree}(v) = E$$

1.2.3 连通性边数

含 V 个顶点的简单无向图中：

- 最少边数为 0；
- 最多边数为

$$\binom{V}{2} = \frac{V(V-1)}{2}$$

- 连通图至少有 $V-1$ 条边；
- 保证无论怎样放边都一定连通，至少需要

$$\binom{V-1}{2} + 1 = \frac{(V-1)(V-2)}{2} + 1$$

原因是：边数最多的非连通图可以由一个含 $V-1$ 个顶点的完全图和一个孤立顶点组成。

1.2.4 邻接矩阵

无权图的邻接矩阵：

$$A[i][j]=\begin{cases}1, & (v_i,v_j)\in E\\ 0, & \text{其他}\end{cases}$$

带权图的邻接矩阵：

$$A[i][j]=\begin{cases}w(i,j), & (v_i,v_j)\in E\\ \text{INF}, & i\neq j \text{ 且无边}\\ 0, & i=j\end{cases}$$

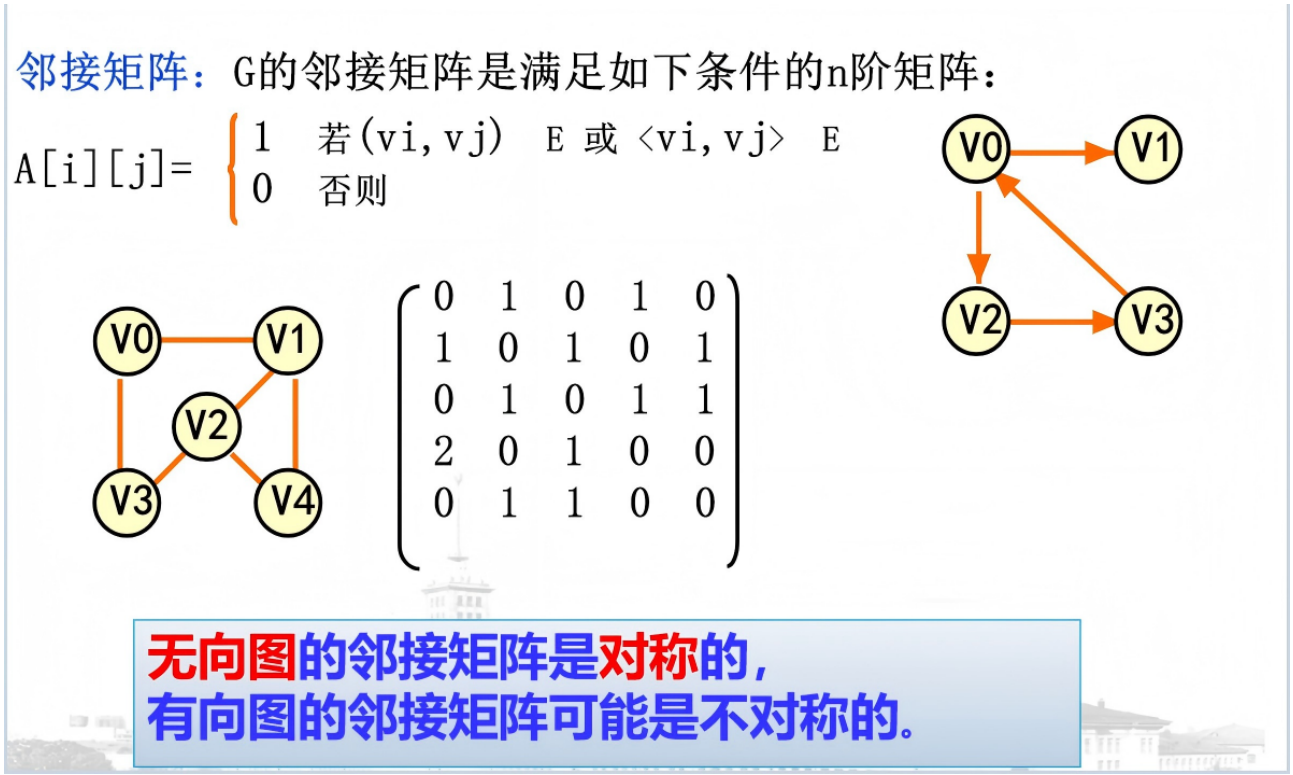


Figure 1: 邻接矩阵与图的对应关系

图 1-1　邻接矩阵与图的对应关系

邻接矩阵的特点：

- 空间复杂度：$O(V^2)$；
- 判断两点是否相邻：$O(1)$；
- 枚举一个顶点的全部邻接点：$O(V)$；
- 无向图的邻接矩阵关于主对角线对称；

- 有向图第 i 行非零项数通常对应顶点 i 的出度（注意：带权邻接矩阵以 `INF` 表示无边时，不能按“非零项数”一般性统计度数——无边项不是 0 而是 `INF`，且可能存在权值为 0 的边；统计度数时应排除 `INF` 项、计入所有出边）。

1.2.5 邻接表

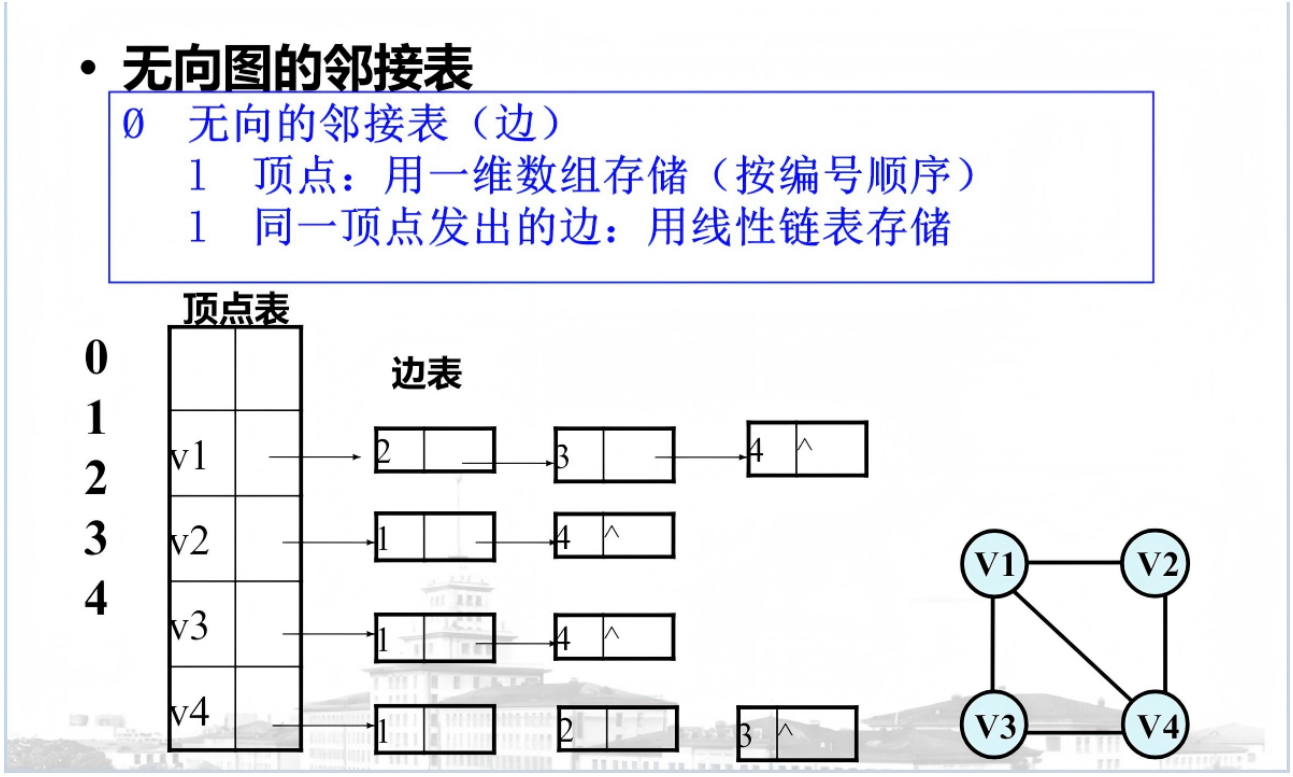


Figure 2: 无向图邻接表

图 1-2　无向图邻接表

邻接表的特点：

- 空间复杂度：$O(V+E)$；
- 枚举顶点 u 的邻接点：$O(\deg(u))$；
- 适合存储稀疏图；
- 有向图邻接表便于求出度；
- 求入度需要扫描全部边，或额外维护 `indegree[]`。

1.2.6 存储结构对比

存储结构	适用图	空间复杂度	查询邻接关系	枚举邻接点
邻接矩阵	稠密图	$O(V^2)$	$O(1)$	$O(V)$
邻接表	稀疏图	$O(V+E)$	最坏 $O(\deg(u))$	$O(\deg(u))$

1.3 【典型题型精析】

题型 1：保证无向图一定连通的最少边数（2025 春往年题）

题干呈现 2025 春填空题：一个包含 7 个顶点、边不重复的无向图，至少需要多少条边，才能确保在顶点间任意分配这些边后，该图始终连通?

解题思路分析 题目问的是“无论怎样连接都连通”，不能直接回答 $V-1$。
$V-1$ 只是构造一个连通图所需的最少边数，并不能保证任意放置这些边都连通。

应从“边数最多的非连通图”反向考虑。

详细解答步骤

1. 为了让非连通图拥有尽可能多的边，应把尽可能多的顶点放在同一个连通分量中。
2. 极端情况是：$V-1$ 个顶点构成完全图，剩余一个顶点孤立。
3. 此时边数为：

$$\binom{V-1}{2}=\frac{(V-1)(V-2)}{2}$$

4. 再增加一条边时，不可能继续保持这种非连通状态。
5. 因此答案为：

$$\boxed{\binom{V-1}{2}+1}$$

代入 $V=7$：

$$\binom{6}{2}+1=\frac{6\times 5}{2}+1=\boxed{16}$$

易错点与总结

- 易错：直接写 $V-1$。
- 区分：
  - “存在一种连法使图连通”对应 $V-1$；
  - “任意连法都保证连通”对应 $\binom{V-1}{2}+1$。

题型 2：由图写邻接矩阵

题干呈现 给出一个带权有向图，要求写出邻接矩阵，并说明矩阵是否对称。

解题思路分析 先确定“行表示弧尾、列表示弧头”，然后逐条读取有向边。不要因为两个顶点之间画了连线就同时填写两个方向。

详细解答步骤

1. 按题目指定顺序排列顶点。
2. 主对角线填写 0。
3. 若存在弧 $v_i \to v_j$，填写权值 $w(i,j)$。
4. 若不存在弧且 $i \neq j$，填写 `INF`。
5. 最后检查每条有向弧是否只填写了一个方向。

易错点与总结

- 有向图邻接矩阵不一定不对称：若每条弧都恰好有反向弧且权值相同，矩阵也可能对称。
- “无向图邻接矩阵一定对称”正确。
- “有向图邻接矩阵一定不对称”错误。

1.4 【自查与巩固建议】

自查提问

1. 无向图所有顶点度数之和为什么等于 $2E$?
2. 有向图邻接矩阵的行和、列和分别对应什么?
3. 为什么邻接矩阵更适合稠密图?
4. 为什么无向图邻接表中一条边通常要存储两次?
5. 有向图的邻接表为什么通常更容易统计出度?

关键词填空

1. 无向图邻接矩阵关于主对角线 _____。
2. 邻接表的空间复杂度为 _____。
3. 含 V 个顶点的生成树具有 _____ 条边。
4. 保证简单无向图一定连通的边数阈值为 _____。

参考答案：对称；$O(V+E)$；$V-1$；$\binom{V-1}{2}+1$。

进一步练习方向

- 由同一张图分别写邻接矩阵、邻接表。
- 根据邻接矩阵计算度、入度、出度。
- 比较矩阵和邻接表上的 $\mathrm{DFS}/\mathrm{BFS}$ 复杂度。

---

2. 图的遍历与生成树

2.1 【考点体系梳理】

本节知识逻辑如下：

图的遍历
├─ DFS
│ ├─ 递归栈或显式栈
│ ├─ 深入与回溯
│ └─ DFS 生成树
└─ BFS
   ├─ 队列
   ├─ 按距离分层
   ├─ 无权最短路径
   └─ BFS 生成树

DFS 与 BFS 的共同点：

- 都需要 visited[]；
- 从一个起点只能访问该起点所在的无向连通分量（对有向图，只能访问从该起点沿有向边可达的顶点集，它既不是弱连通分量也不是强连通分量）；
- 遍历非连通图时，需要在外层枚举全部顶点；
- 邻接表实现的时间复杂度均为 $O(V+E)$。

主要区别：

- DFS 强调“深入—回退”；
- BFS 强调“发现—入队—分层扩展”。

2.2 【核心概念与公式】

2.2.1 DFS

图 2-1　DFS 的深入与回溯

伪代码：

```
void DFS(Graph G, int u) {
    visited[u] = true;
    visit(u);
    for (int v : Adj[u]) {
        if (!visited[v]) {
            DFS(G, v);
        }
    }
}
```

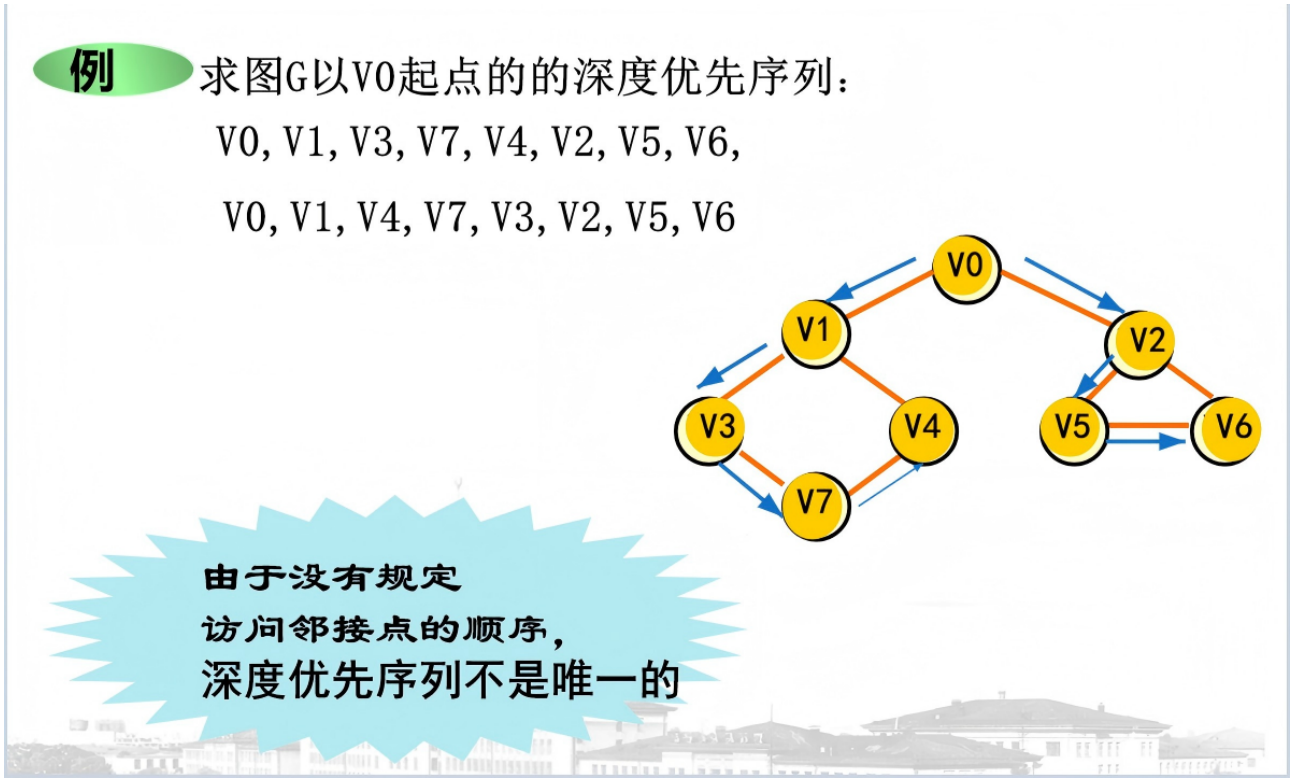


Figure 3: DFS 过程

2.2.2 BFS

图 2-2 BFS 的队列推进

伪代码：

```
void BFS(Graph G, int s) {
    for (int v = 0; v < V; ++v) visited[v] = false;
    queue<int> q;
    visited[s] = true;
    q.push(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        visit(u);

        for (int v : Adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
```

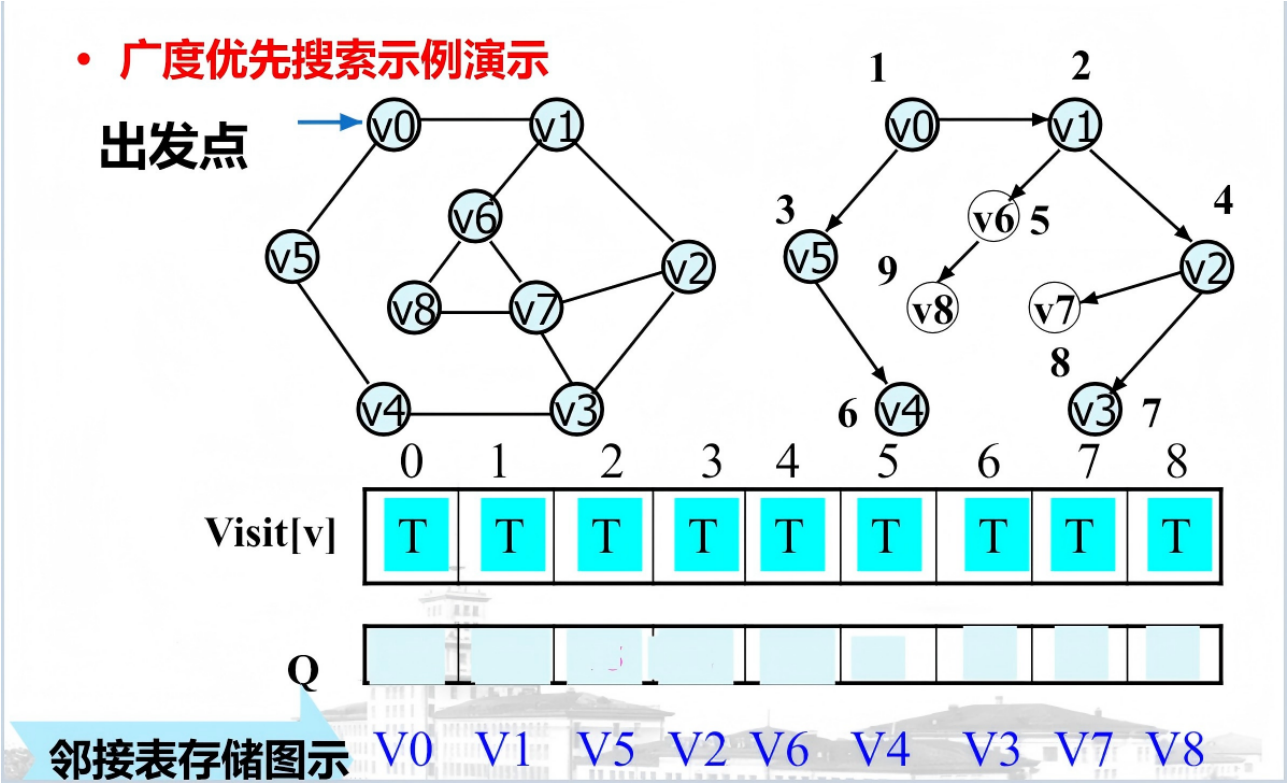


Figure 4: BFS 过程

}

必须在入队时标记。若在出队时才标记，同一个顶点可能被多个前驱重复加入队列。

2.2.3 BFS 距离分层

图 2-3　BFS 的无权最短距离分层 无权图中：

$$\text{dist}[v] = \text{dist}[u] + 1$$

其中 u 是第一次发现 v 的顶点。

2.2.4 遍历生成树

图 2-4　DFS/BFS 生成树

顶点第一次被发现时使用的边称为树边。若原无向图连通，生成树：

- 包含全部 V 个顶点；
- 具有 $V - 1$ 条树边；
- 连通且无环。

2.2.5 复杂度

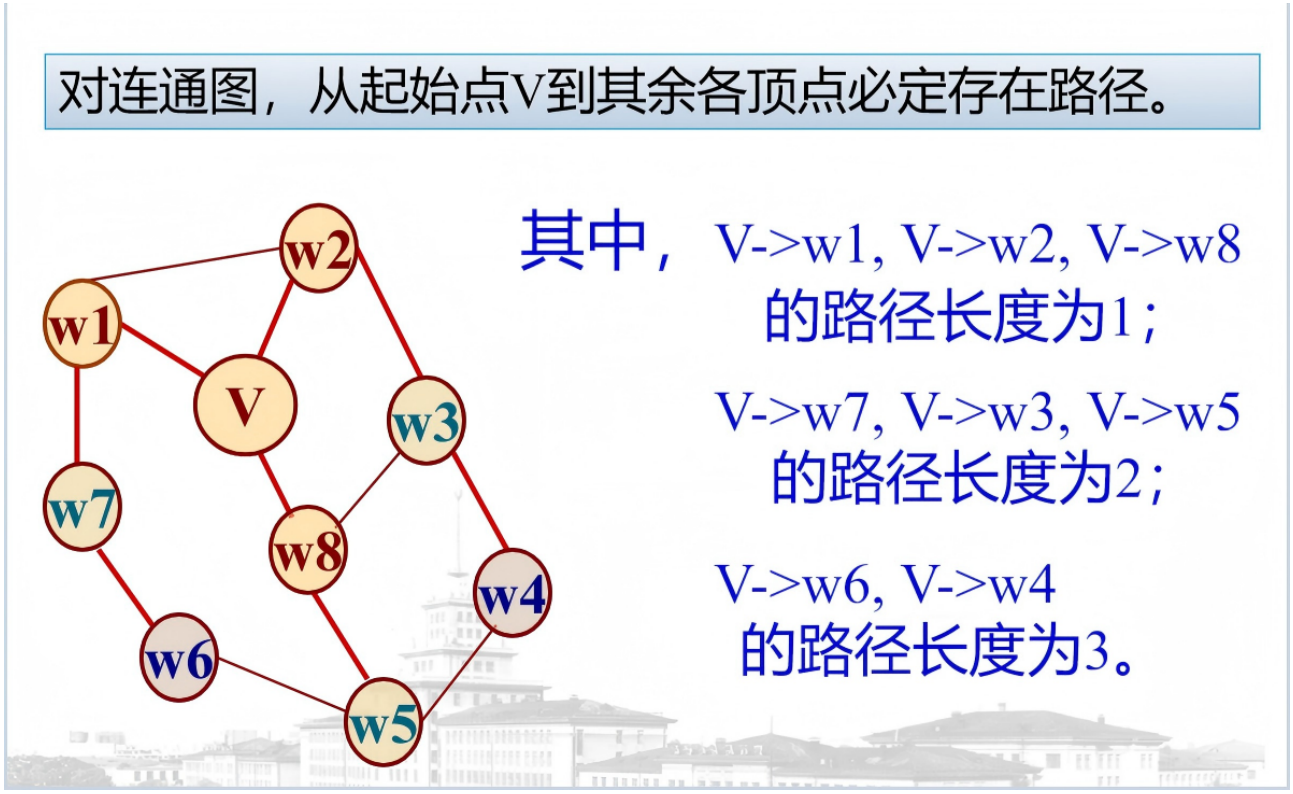


Figure 5: BFS 距离分层

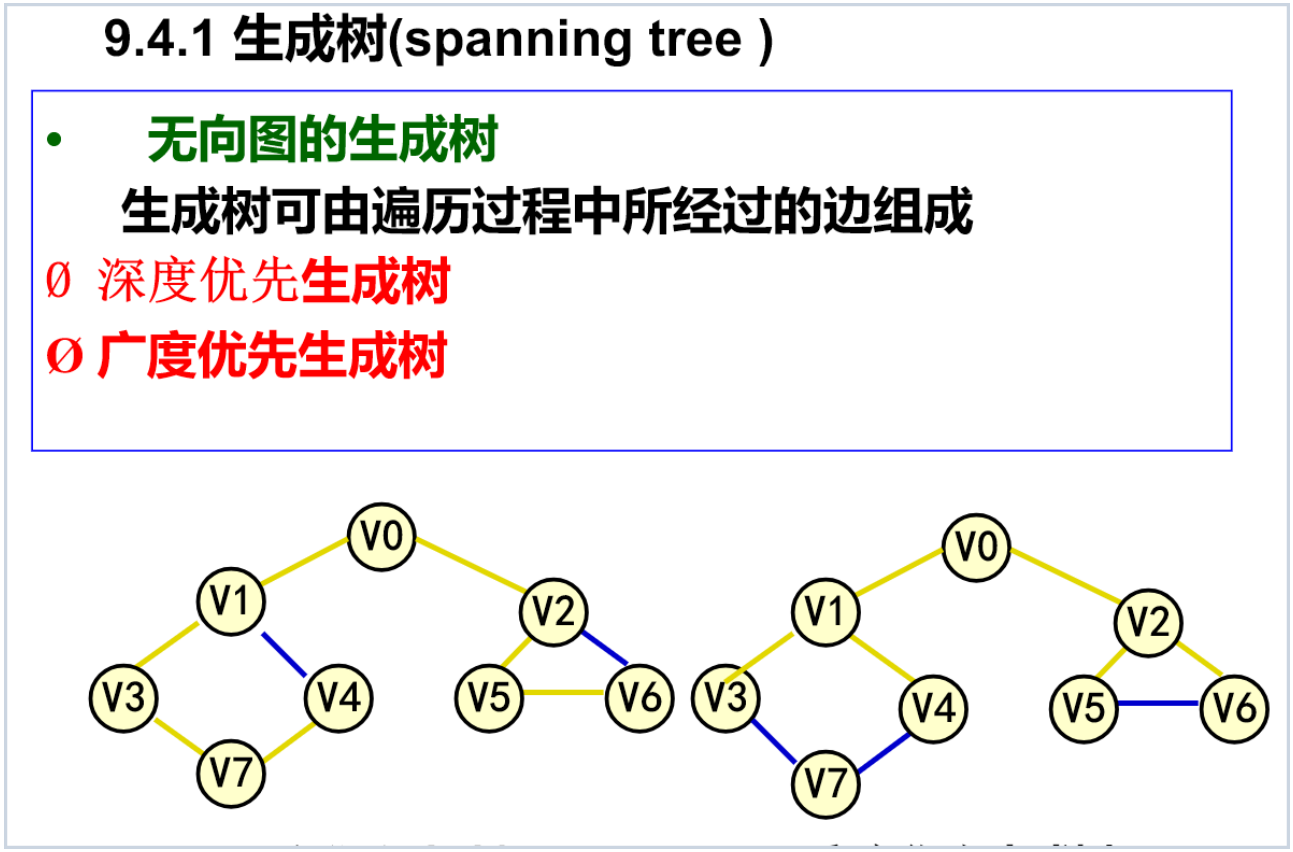


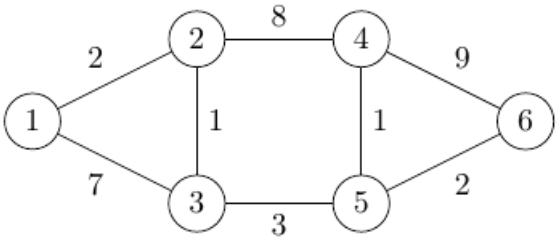
Figure 6: DFS 与 BFS 生成树

存储结构	DFS	BFS
邻接矩阵	$O(V^2)$	$O(V^2)$
邻接表	$O(V+E)$	$O(V+E)$

2.3 【典型题型精析】

题型 1：邻接矩阵 + BFS 队列峰值（2025 春往年题）

2. 根据给出的无向图回答下列问题。



（1）用三元顺序组表示该图的邻接矩阵。

（2）从结点 1 开始进行广度优先遍历，写出遍历序列。若广度优先遍历的实现是先入队列，则遍历过程中队列最长的时候有多长?

（3）求最小生成树，并说明算法思想。

Figure 7: 2025 春图综合题

题干呈现

往年题 2-1　　2025 春

对图使用三元顺序组表示邻接矩阵；从顶点 1 出发，按编号升序访问邻接点，写出 BFS 序列，并求“先出队、后入队”时队列的最大长度。

解题思路分析　该题不能只写最终序列。队列峰值与操作定义有关，必须逐轮写出队列状态。

详细解答步骤　由题图得到无向边：

```
(1,2,2), (1,3,7), (2,3,1), (2,4,8),
(3,5,3), (4,5,1), (4,6,9), (5,6,2)
```

标准三元组表应记录矩阵的**全部非零项**：无向图的每条边对应两个对称位置，应写出两项 (i,j,w) 与 (j,i,w)；只有题目明确采用对称压缩（“只记录上三角”）的约定时，才能省略对称项。下面的写法即题目采用上三角压缩约定的形式：

(1,2,2), (1,3,7), (2,3,1), (2,4,8),
(3,5,3), (4,5,1), (4,6,9), (5,6,2)

若题目要求把无向边的两个方向都展开，则每个三元组还要补写交换行、列下标后的对称项。按升序访问邻接点：

操作	队列状态	新访问顶点
初始化	[1]	1
1 出队，2、3 入队	[2,3]	2、3
2 出队，4 入队	[3,4]	4
3 出队，5 入队	[4,5]	5
4 出队，6 入队	[5,6]	6
5 出队	[6]	无
6 出队	[]	无

因此：

BFS 序列：1,2,3,4,5,6
队列最大长度：2

易错点与总结

- 顶点入队时立即标记，防止重复入队。
- 必须说明邻接点访问顺序。
- 队列峰值题必须先确认题目采用“先出后入”还是“先入后出”的统计口径。

题型 2：遍历非连通图

题干呈现 写出可以遍历非连通图全部顶点的 DFS 框架。

解题思路分析 一次 DFS(s) 只能覆盖 s 所在的无向连通分量（对有向图则是从 s 沿有向边可达的顶点集，不等于弱连通分量或强连通分量），因此必须在外层扫描所有顶点。

详细解答步骤

```
void DFSTraverse(Graph G) {
    for (int v = 0; v < V; ++v) {
        visited[v] = false;
    }

    for (int v = 0; v < V; ++v) {
        if (!visited[v]) {
            DFS(G, v);
        }
```

```
    }
}
```

外层每启动一次新的 DFS，就发现一个新的连通分量（这一结论仅对无向图成立）。

易错点与总结

- 只从顶点 0 调用一次 DFS，不能保证遍历非连通图。
- 无向图的连通分量数可以通过外层成功启动 DFS 的次数统计（有向图的外层 DFS 次数不直接给出强连通分量数）。

2.4 【自查与巩固建议】

自查提问

1. 为什么 BFS 要在入队时标记?
2. DFS 的递归调用栈保存了什么信息?
3. 一次 DFS 为什么恰好访问无向图的一个连通分量?
4. BFS 生成树为什么能反映无权最短距离?
5. 邻接点次序改变后，哪些结果可能改变?

关键词填空

1. DFS 的核心辅助结构是 _____。
2. BFS 的核心辅助结构是 _____。
3. 连通无向图的遍历生成树具有 _____ 条边。
4. 邻接表下 DFS/BFS 的复杂度为 _____。

参考答案：栈；队列；$V-1$；$O(V+E)$。

进一步练习方向

- 手写不同邻接点顺序下的 DFS/BFS 序列。
- 统计 BFS 每轮队列长度。
- 根据遍历过程中第一次发现顶点的边画生成树。

---

3. 最小生成树

3.1 【考点体系梳理】

最小生成树解决的问题是：

> 在连通无向带权图中选择 $V-1$ 条边，使全部顶点连通且总权值最小。

知识逻辑：

生成树
├─ 连通
├─ 无环
└─ V-1 条边

最小生成树
├─ Prim：从顶点集合向外扩张
└─ Kruskal：从全局最小边开始合并分量

两种算法都属于贪心算法，但贪心对象不同：

- Prim 每轮选择“连接树内与树外”的最小边。
- Kruskal 每轮选择“不会形成环”的全局最小边。

3.2 【核心概念与公式】

3.2.1 生成树性质

含 V 个顶点的生成树：

$$|E_T| = V - 1$$

并满足：

- 增加任意一条边都会形成环；
- 删除任意一条边都会断开；
- 任意两个顶点之间有且只有一条简单路径。

3.2.2 Prim

Prim 维护：

- `used[v]`：顶点是否已进入生成树；
- `lowcost[v]`：树外顶点到当前树的最小连接代价；
- `parent[v]`：产生该最小代价的树内顶点。

复杂度：

- 邻接矩阵：$O(V^2)$；
- 优先队列与邻接表：$O(E \log V)$。

3.2.3 Kruskal

Kruskal 的步骤：

1. 按边权从小到大排序。
2. 初始化并查集，每个顶点自成集合。
3. 依次考察边 (u, v)。
4. 若 `find(u) != find(v)`，选择该边并合并集合。
5. 选满 $V - 1$ 条边后停止。

复杂度主要来自排序：

$$O(E \log E)$$

3.2.4 算法对比

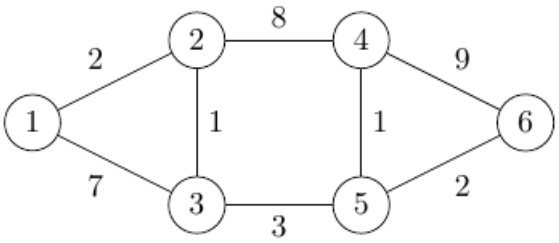
比较项	Prim	Kruskal
贪心对象	当前树到树外的最小边	全局最小且不成环的边
防环机制	新加入一个树外顶点	并查集
更适合	稠密图	稀疏图
核心状态	`lowcost[]`	边排序、`parent[]`

3.3 【典型题型精析】

题型 1：Kruskal 手算最小生成树（2025 春往年题）

题干呈现 2025 春，使用 Kruskal 算法求最小生成树，并写出选边顺序与总权值。

2. 根据给出的无向图回答下列问题。



（1）用三元顺序组表示该图的邻接矩阵。

（2）从结点 1 开始进行广度优先遍历，写出遍历序列。若广度优先遍历的实现是先入队列，则遍历过程中队列最长的时候有多长？

（3）求最小生成树，并说明算法思想。

Figure 8: 2025 春最小生成树题图

解题思路分析　先按边权排序，再判断边的两个端点是否属于不同连通分量。不能只凭图形“看起来没有环”。

详细解答步骤　先按权值排序：

```
(2,3,1), (4,5,1),
(1,2,2), (5,6,2),
(3,5,3), (1,3,7),
(2,4,8), (4,6,9)
```

逐边处理：

当前边	是否选取	原因
$(2,3,1)$	选	合并顶点 2、3 所在分量
$(4,5,1)$	选	合并顶点 4、5 所在分量
$(1,2,2)$	选	把顶点 1 接入 $\{2,3\}$
$(5,6,2)$	选	把顶点 6 接入 $\{4,5\}$
$(3,5,3)$	选	合并两大分量，全部顶点连通

已经选满 $V-1=5$ 条边，停止。最小生成树边集为：

$$\{(2,3),(4,5),(1,2),(5,6),(3,5)\}$$

总权值：

$$1+1+2+2+3=\boxed{9}$$

易错点与总结

- 权值相同的边可能有多种合法选取顺序。
- 最小生成树不一定唯一，但不同最小生成树的总权值相同。
- $\mathrm{Kruskal}$ 处理的是无向图；有向图不能直接套用普通 MST。

题型 2：判断最小生成树是否唯一

题干呈现　一个连通无向带权图存在若干条权值相同的边，判断其最小生成树是否一定唯一。

解题思路分析　“存在相同权值边”并不能直接推出 MST 不唯一。关键是这些同权边是否能在保持最小总权值的前提下互换。

详细解答步骤

1. 运行 Prim 或 Kruskal 得到一棵 MST。
2. 对未被选中的边 (u, v, w)，观察 MST 中 u 到 v 的唯一路径。
3. 若路径上存在一条权值也为 w 的边，则可以交换，得到另一棵相同总权值的 MST。
4. 若所有未选边都不能完成等权交换，则 MST 唯一。

易错点与总结

- “边权互不相同”可以推出 MST 唯一。
- “存在相同边权”不能直接推出 MST 不唯一。

3.4 【自查与巩固建议】

自查提问

1. Prim 和 Dijkstra 的状态更新看起来相似，本质目标有什么不同?
2. Kruskal 为什么必须使用并查集?
3. 生成树为什么恰有 $V - 1$ 条边?
4. 权值相同的边一定导致 MST 不唯一吗?

关键词填空

1. Prim 每轮选择连接 _____ 与 _____ 的最小边。
2. Kruskal 使用 _____ 判断加入边后是否成环。
3. 最小生成树只适用于连通 _____ 带权图。

参考答案：树内、树外；并查集；无向。

进一步练习方向

- 对同一张图分别执行 Prim 与 Kruskal。
- 写出 Kruskal 每一步的并查集状态。
- 构造一个具有两棵不同 MST 的图，并说明两棵树的总权值相同。

---

4. 最短路径

4.1 【考点体系梳理】

最短路径算法的选择首先取决于图的条件：

最短路径
├─ 无权单源：BFS
├─ 非负权单源：Dijkstra

├─ 可含负边的单源：Bellman-Ford
├─ DAG 单源：拓扑序松弛
└─ 所有点对：Floyd

所有算法的共同核心是“松弛”，但不同算法对松弛顺序和状态定型时机有不同要求。

4.2 【核心概念与公式】

4.2.1 松弛

- 最短路径算法的核心技术是松弛
- 以 表示顶点 出发到顶点 的边的权值，以 表示当前从起点到顶点 的路径权值

```
1.  Relax(u,v,w(u,v)) {
2.      If (d[v] > d[u] + w(u,v)) Then
3.          d[v] = d[u] + w(u,v);
4.  }
```

Figure 9: 松弛操作

图 4-1　最短路径的核心操作：松弛

若：

$$\text{dist}[u] + w(u, v) < \text{dist}[v]$$

则更新：

$$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$$

同时可记录：

$$\text{prev}[v] \leftarrow u$$

4.2.2 Dijkstra

适用条件：所有边权非负。

每轮：

1. 从未定型顶点中选择 dist 最小的顶点 u。
2. 将 u 加入已定型集合 S。
3. 松弛 u 的全部出边。

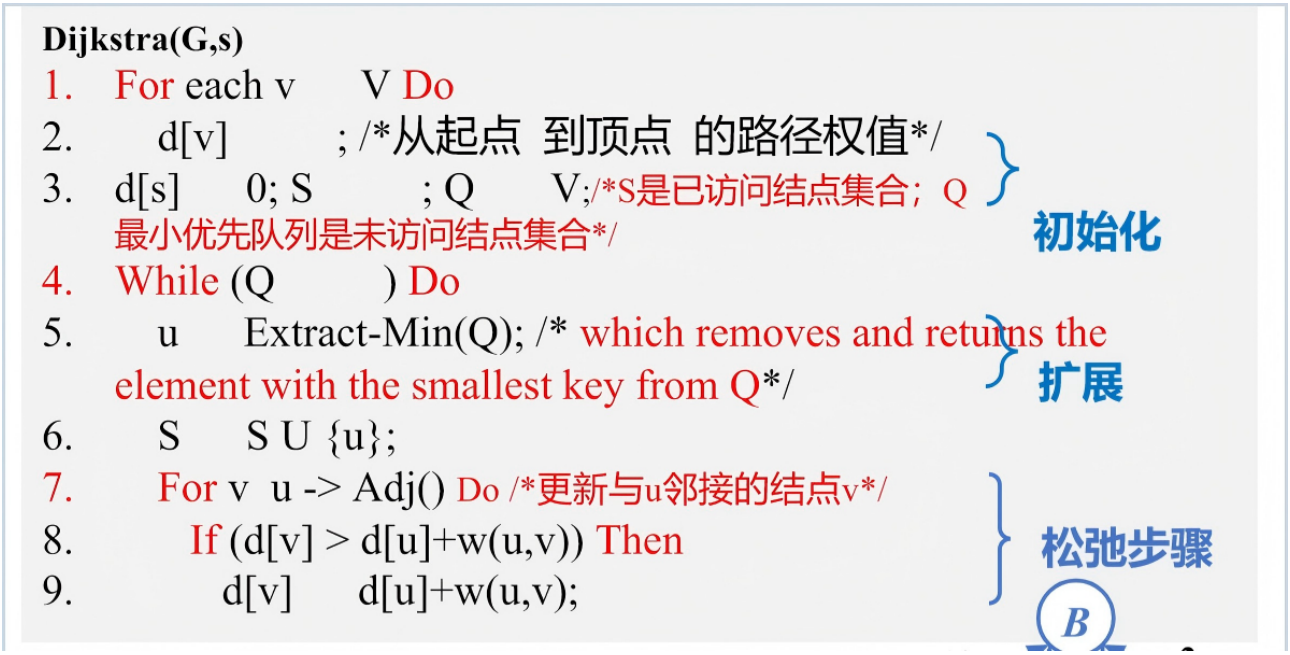


Figure 10: Dijkstra 初始过程

图 4-2 Dijkstra

复杂度：

- 邻接矩阵：$O(V^2)$；
- 最小堆与邻接表：$O(E \log V)$。

4.2.3 Bellman-Ford

适用条件：

- 允许负权边；
- 若要求有限最短路，则不能存在从源点可达的负权回路。

核心步骤：

```
for (int i = 1; i <= V - 1; ++i) {
    bool changed = false;
    for (Edge e : edges) {
        if (dist[e.u] != INF &&
            dist[e.v] > dist[e.u] + e.w) {
            dist[e.v] = dist[e.u] + e.w;
            changed = true;
        }
    }
```

```
    if (!changed) break;
}
```

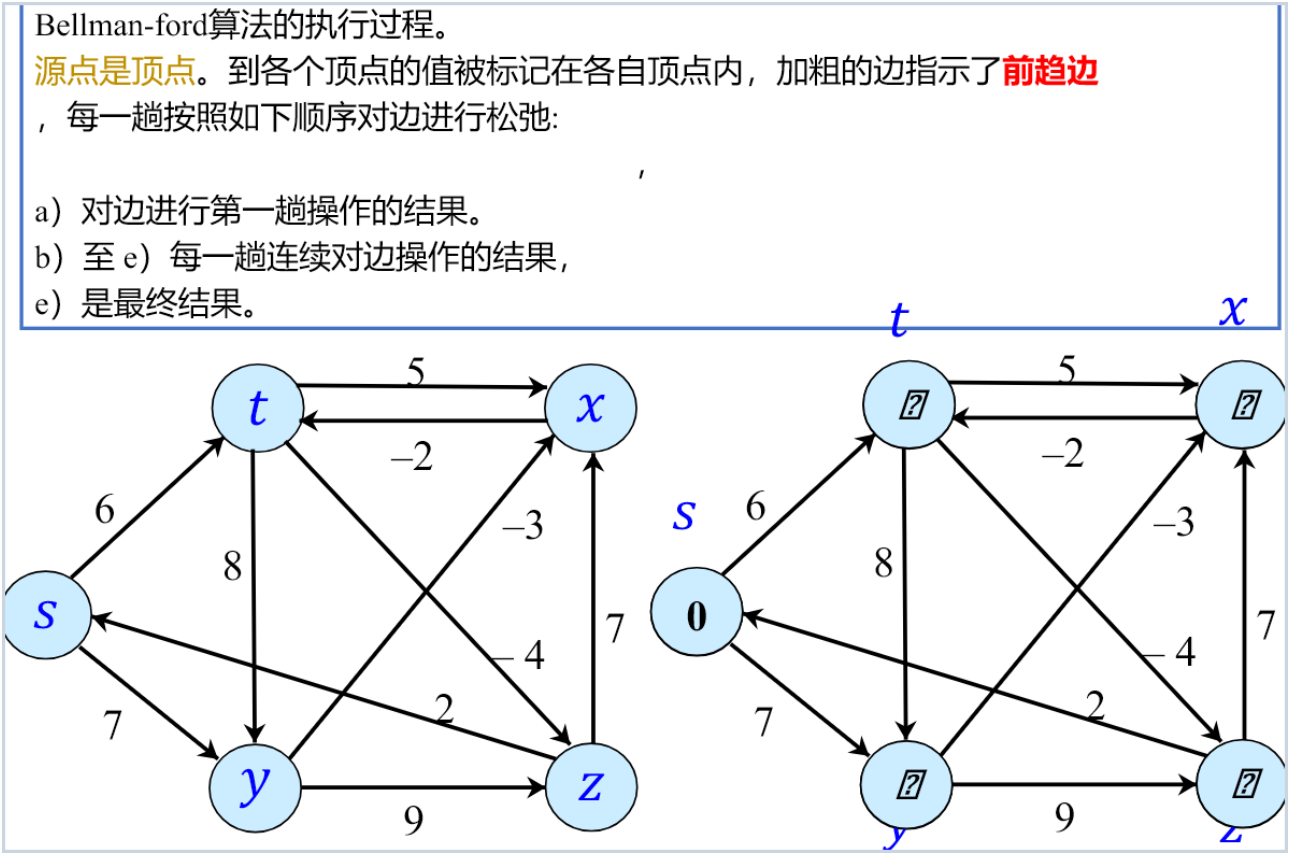


Figure 11: Bellman-Ford 前期状态

图 4-3 Bellman-Ford 的轮次变化

第 k 轮后，可以保证得到“至多使用 k 条边”的最短路径。简单路径最多包含 $V-1$ 条边，因此最多需要 $V-1$ 轮。

负环判定：

完成 $V-1$ 轮后再扫描所有边，如果仍能松弛，则存在从源点可达的负权回路。

4.2.4 Floyd

Floyd 求所有顶点对之间的最短路径。

递推式：

$$D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)$$

压缩为二维矩阵：

```
for (int k = 0; k < V; ++k)
    for (int i = 0; i < V; ++i)
```

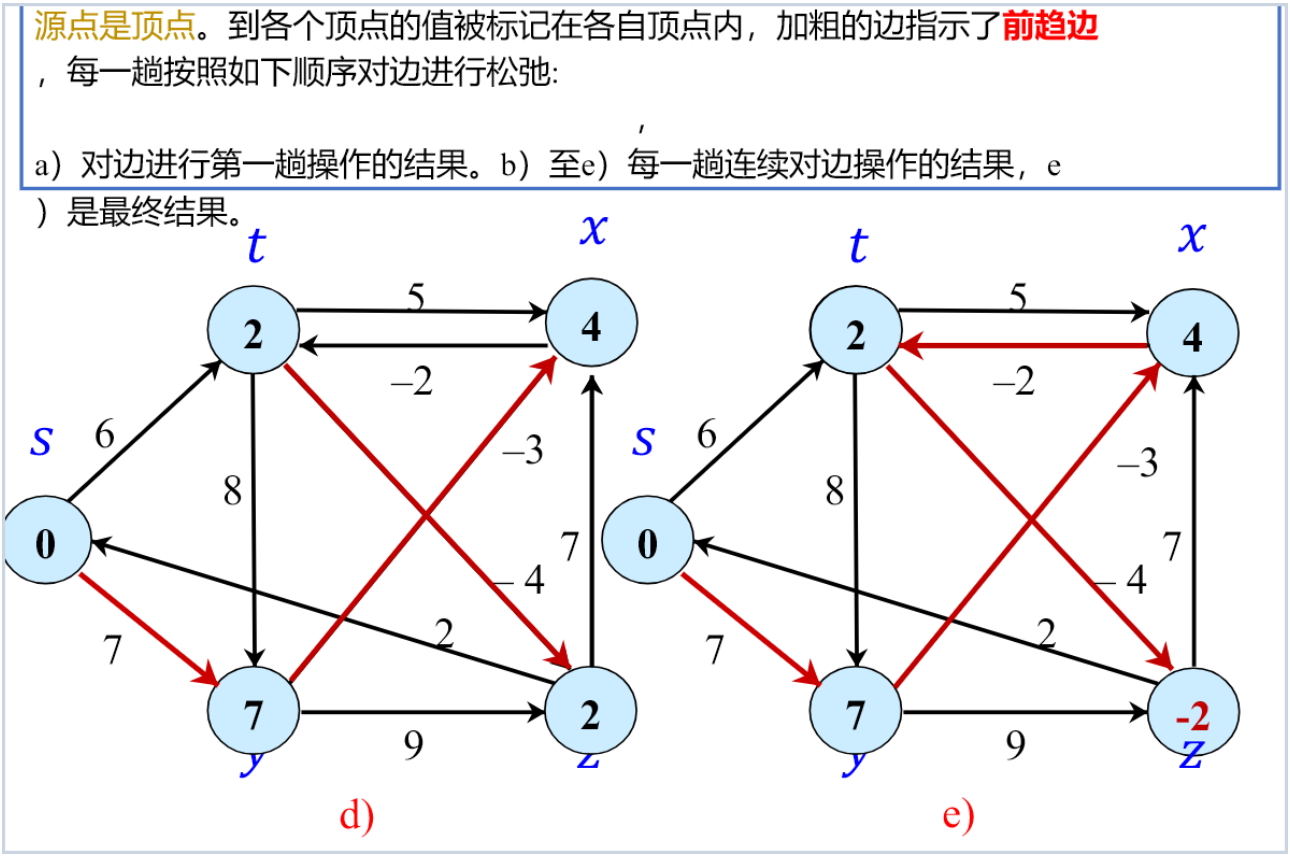


Figure 12: Bellman-Ford 末轮状态

```
for (int j = 0; j < V; ++j)
    D[i][j] = min(D[i][j], D[i][k] + D[k][j]);
```

k 必须位于最外层。

复杂度：

$$T(V) = O(V^3), \qquad S(V) = O(V^2)$$

适用条件与注意：

- Floyd 可以正确处理负权边；但若图中存在负权回路，则相关点对之间不存在有限最短路，算法结果无意义。实现中可先做 V 轮后检查 $D[i][i] < 0$ 来探测负环，或用 Bellman-Ford 检测。
- 以有限大数 `INF` 表示不可达时，松弛语句必须先判断两段都可达再做加法，即仅在 `D[i][k] != INF && D[k][j] != INF` 时计算 `D[i][k] + D[k][j]`；否则 `INF + w` 可能溢出，或负边把不可达误判为可达。

4.2.5 A* 搜索

A* 使用评价函数：

$$f(n) = g(n) + h(n)$$

其中：

- $g(n)$：从起点到结点 n 的当前实际代价；
- $h(n)$：从结点 n 到目标的估计代价；
- $f(n)$：经过结点 n 到达目标的估计总代价。

若要保证 A^* 得到最优解，需要区分搜索方式：

- **树搜索**：启发函数可采纳（$0 \le h(n) \le h^*(n)$，即目标真实剩余代价的下界）通常已足够；
- **图搜索**：若已扩展（关闭）的结点不再重新打开，还需要**一致性（单调性）**条件 $h(n) \le c(n, n') + h(n')$（对所有边 $(n, n') \in E$），或在找到更优 g 时允许重开结点；通常还假定边代价非负。

这种启发函数称为可采纳的或乐观的估计。$h(n)$ 越接近真实剩余代价，通常扩展的结点越少；当 $h(n) = 0$ 时，A^* 退化为一致代价搜索（按 g 取最小扩展，即 Dijkstra 的优先队列形式）。

4.3 【典型题型精析】

题型 1：A^* 评价函数判断（2026 春往年题）

9. 在图示带权有向图上应用 A* 算法。设启发函数 h 按各结点所有出边权值中的最小值取值。以下说法不正确的是（ ）

A. $h(v_5) = 2$

B. $f(v_2) = 13$

C. 从 s 到 t 的最优路径为 $s \to v_1 \to v_4 \to t$

D. 启发函数 h 应当是到目标代价的下界估计，即一种“乐观”的估计

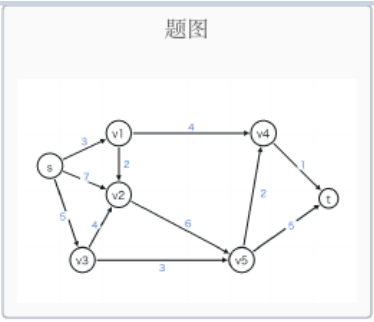


Figure 13: 2026 春 A^* 选择题

题干呈现

往年题 4-1　　2026 春

解题思路分析　先按定义计算每个顶点的 h，再计算从 s 到该顶点的最小已知代价 g，最后使用 $f = g + h$。不能把边权、g、h 和 f 混为一谈。

详细解答步骤

1. 顶点 v_5 的出边权值为 2 和 5，因此：

$$h(v_5) = \min(2, 5) = 2$$

故 A 正确。

2. 到达 v_2 的较短路径为 $s \to v_1 \to v_2$：

$$g(v_2) = 3 + 2 = 5$$

而 v_2 的出边只有 $v_2 \to v_5(6)$，所以：

$$h(v_2) = 6$$

因此：

$$f(v_2) = g(v_2) + h(v_2) = 5 + 6 = 11$$

不是 13，故 B 不正确。

3. 路径 $s \to v_1 \to v_4 \to t$ 的代价为：

$$3 + 4 + 1 = 8$$

与其他可行路径比较后，它是最优路径，故 C 正确。

4. 为保证最优性，h 应是到目标真实代价的下界，即乐观估计，故 D 正确。

最终答案：

$$\boxed{\mathrm{B}}$$

易错点与总结

- g 是已经走过的实际代价，h 是尚未走过部分的估计代价。
- 计算 $f(v)$ 前，要先更新出当前更小的 $g(v)$。
- “乐观”表示不高估剩余代价，不是越大越好。
- A^* 在 2025 春与 2026 春均出现，属于需要保留的客观题考点。

题型 2：负权边为什么会使 Dijkstra 失效（2025 秋往年题）

题干呈现

往年题 4-2　　2025 秋

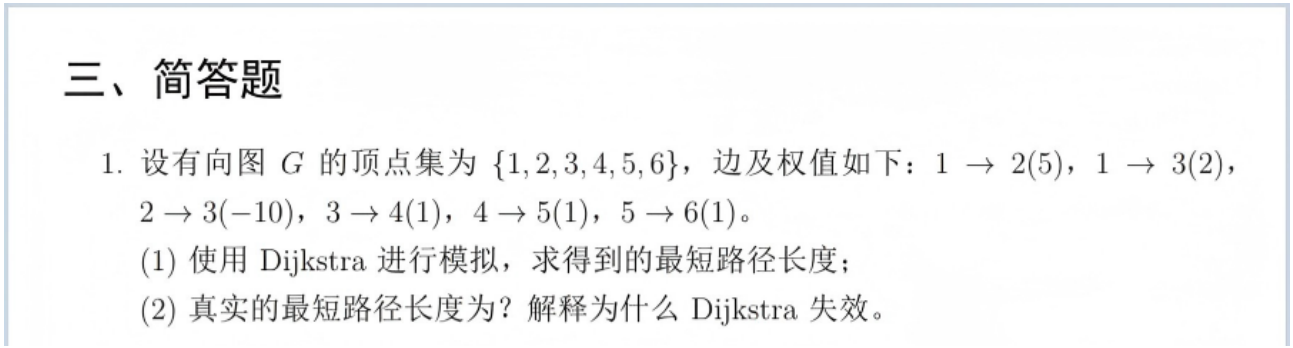


Figure 14: 2025 秋负权 Dijkstra 题

解题思路分析 Dijkstra 依赖一个关键前提：

当前未定型顶点中 `dist` 最小的顶点一旦被选出，其最短距离以后不可能再下降。

负权边会破坏这个前提。

详细解答步骤

1. 初始化：

```
dist[1]=0
dist[2]=5
dist[3]=2
```

2. Dijkstra 先选顶点 3，因为 `dist[3]=2`。
3. 若顶点 3 被定型，算法认为其最短距离已经确定。
4. 之后顶点 2 被处理时，边 $2 \to 3$ 会产生：

$$\text{dist}[2] + (-10) = 5 - 10 = -5$$

5. 真实的顶点 3 最短距离应为 -5，但它此前已经被错误定型。
6. 真实最短路径为：

`1→2→3→4→5→6`

总长度：

$$5 - 10 + 1 + 1 + 1 = -2$$

易错点与总结

- Dijkstra 的限制是“不能存在负权边”，不只是“不能存在负环”。
- 没有负环并不能保证 Dijkstra 正确。
- 含负边且无负环时，应使用 Bellman-Ford；若图同时是 DAG，也可以按拓扑序松弛。

题型 3：Bellman-Ford 负环判定

题干呈现　完成 $V-1$ 轮松弛后，如何判断是否存在从源点可达的负权回路?

解题思路分析　不含负环时，最短简单路径最多只有 $V-1$ 条边，因此 $V-1$ 轮后所有有限最短距离应稳定。

详细解答步骤

1. 正常执行至多 $V-1$ 轮全部边松弛。
2. 再扫描一遍所有边 (u,v,w)。
3. 只考察 `dist[u] != INF` 的边。
4. 若仍满足：

$$\mathrm{dist}[v] > \mathrm{dist}[u] + w$$

则存在从源点可达的负权回路。

易错点与总结

- 必须检查负环是否从源点可达。
- 图中存在一个完全不可达的负环，不会影响该源点的单源最短路。

4.2.6 Floyd 路径恢复

在 Floyd 递推的同时维护路径矩阵。一种常用约定是 $P[i][j]$ 记录 i 到 j 的最短路径上的**后继结点**(或“第一个中间点”)：

$$P[i][j] = \begin{cases} k, & \text{经} k \text{ 松弛后更新,} \\ -1, & i \text{ 到} j \text{ 不可达或} i = j \end{cases}$$

初始化：$P[i][j] = j$ (直接后继)；当 `D[i][k] + D[k][j]` 更新 `D[i][j]` 时，同步令 $P[i][j] = P[i][k]$。

路径恢复：从 i 出发反复读取后继，直到到达 j。若以中间点 k 作为分割点，则路径应分解为：

`i` 到 `k` 的路径 `+ k +` `k` 到 `j` 的路径

易错点与总结

- 必须明确 $P[i][j]$ 存的是中间点、前驱还是后继。
- 不同定义对应不同恢复代码，不能混用。

4.4 【自查与巩固建议】

自查提问

1. Dijkstra 为什么要求边权非负?
2. Bellman-Ford 第 k 轮保证了什么?
3. Floyd 为什么必须把 k 放在最外层?
4. 无权图为什么优先使用 BFS?
5. 如何判断负权回路是否从源点可达?

算法选择自测

场景	应选算法
社交网络中求最少转发层数	BFS
有向图中存在负边但无负环	Bellman-Ford
非负权单源最短路径	Dijkstra
求全部顶点对最短路径	Floyd
DAG 中求单源最短路径	拓扑序松弛

进一步练习方向

- 为 Dijkstra 手写 `S`、`u`、`dist`、`prev` 轮次表。
- 为 Bellman-Ford 写出每一轮后的距离数组。
- 对三顶点图手算 Floyd 每一轮矩阵。

---

5. 拓扑排序与关键路径

5.1 【考点体系梳理】

本节知识关系:

```
有向无环图 DAG
├─ AOV 网
│   └─ 拓扑排序: 解决先修关系、判断有向环
└─ AOE 网
    └─ 关键路径: 计算工程最短完成时间
        ├─ ve: 事件最早发生时间
        ├─ vl: 事件最迟发生时间
        ├─ e: 活动最早开始时间
        ├─ l: 活动最迟开始时间
        └─ l-e=0: 关键活动
```

拓扑排序是关键路径的前置步骤：

- 正向拓扑序用于计算 `ve`；
- 逆拓扑序用于计算 `vl`。

5.2 【核心概念与公式】

5.2.1 拓扑排序

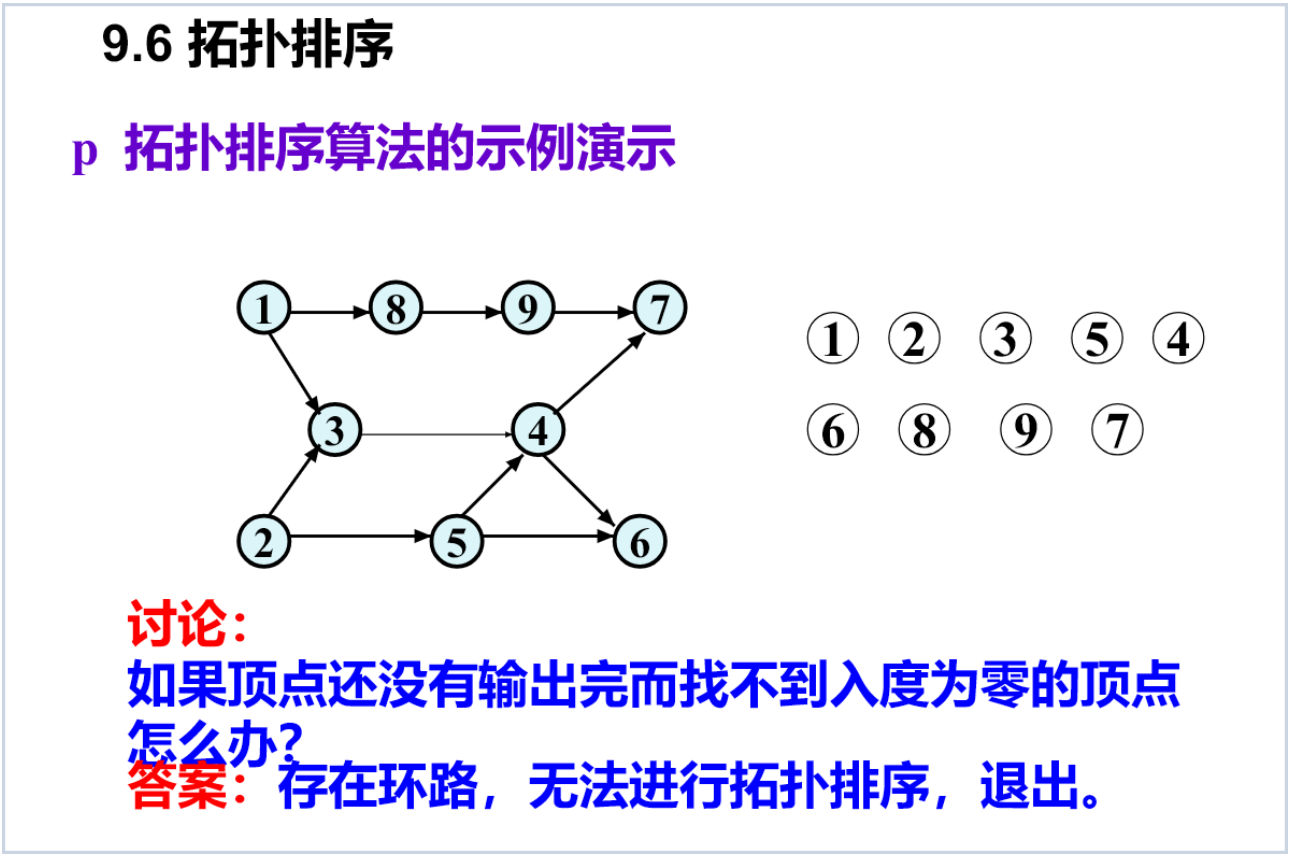


Figure 15: 拓扑排序过程

图 5-1　拓扑序列生成与有向环判定

基本步骤：

1. 统计所有顶点的入度。
2. 将全部入度为 0 的顶点加入栈或队列。
3. 每次取出一个零入度顶点并输出。
4. 删除它的全部出边。
5. 邻接点入度减到 0 时加入容器。
6. 若最终输出顶点数小于 V，图中存在有向环。

图 5-2　邻接表、入度数组和栈

复杂度：

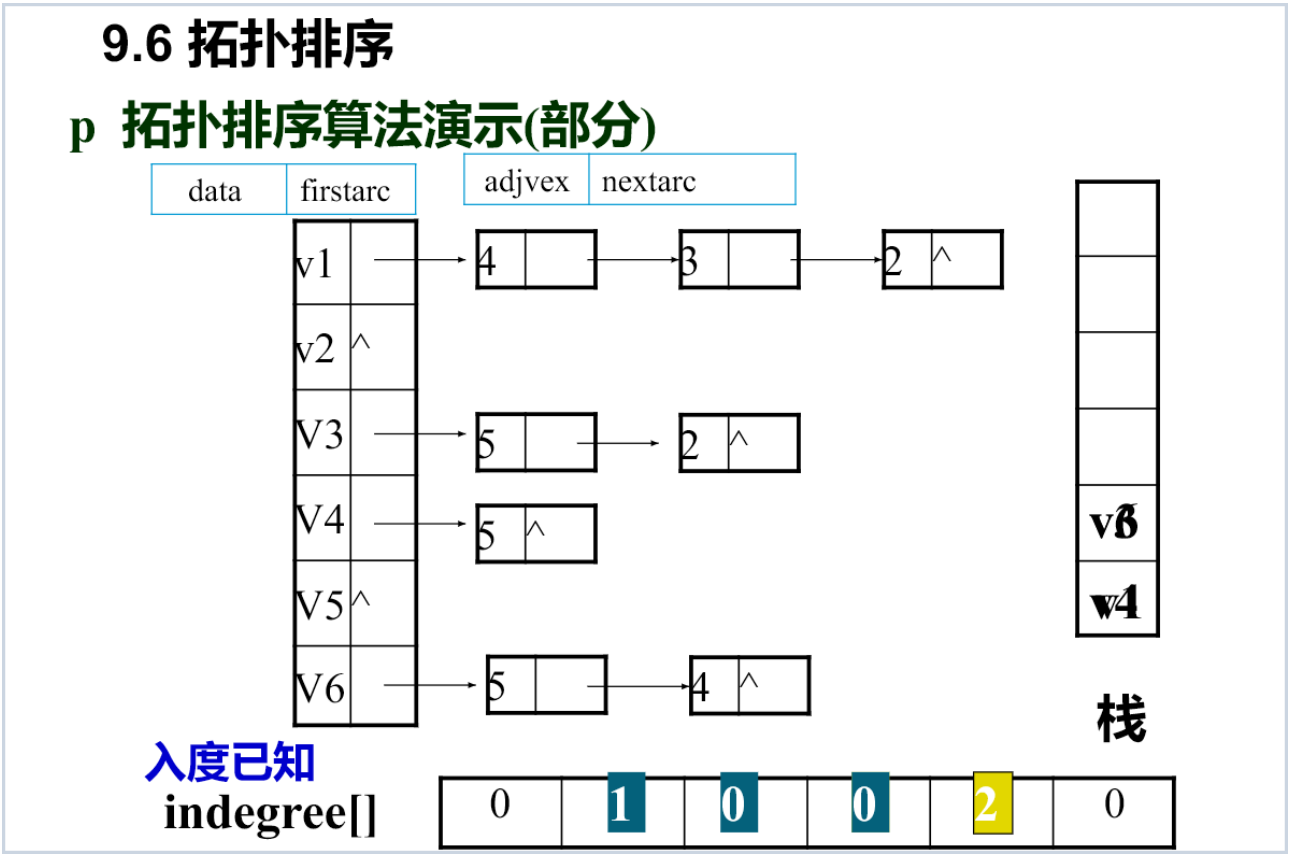


Figure 16: 拓扑排序存储状态

$$O(V+E)$$

5.2.2 事件最早发生时间

图 5-3　ve 按拓扑序取最大值

对于活动边 (u,v):

$$ve[v] = \max_{(u,v)\in E}\{ve[u] + w(u,v)\}$$

5.2.3 事件最迟发生时间

图 5-4　vl 按逆拓扑序取最小值

先令汇点 t:

$$vl[t] = ve[t]$$

再按逆拓扑序计算:

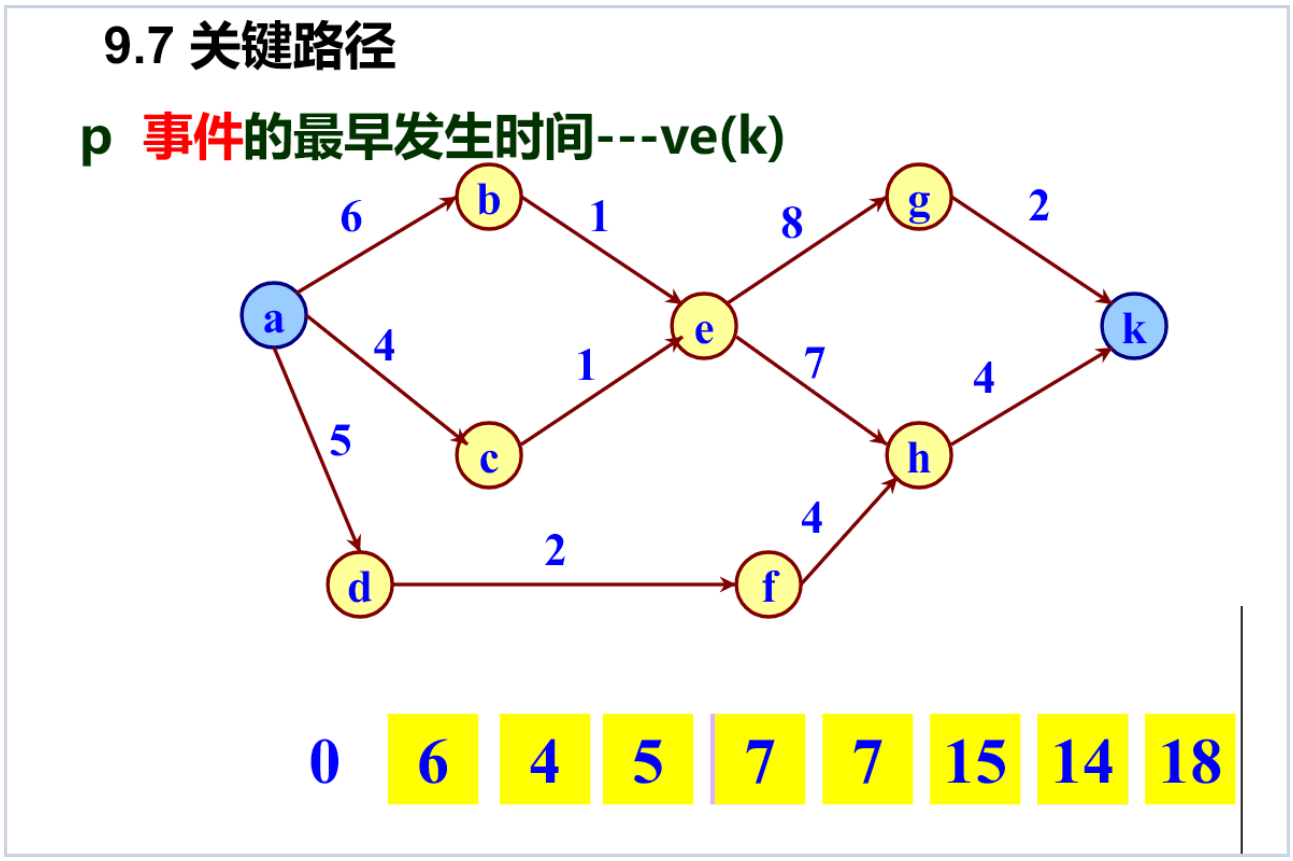


Figure 17: ve 正向计算

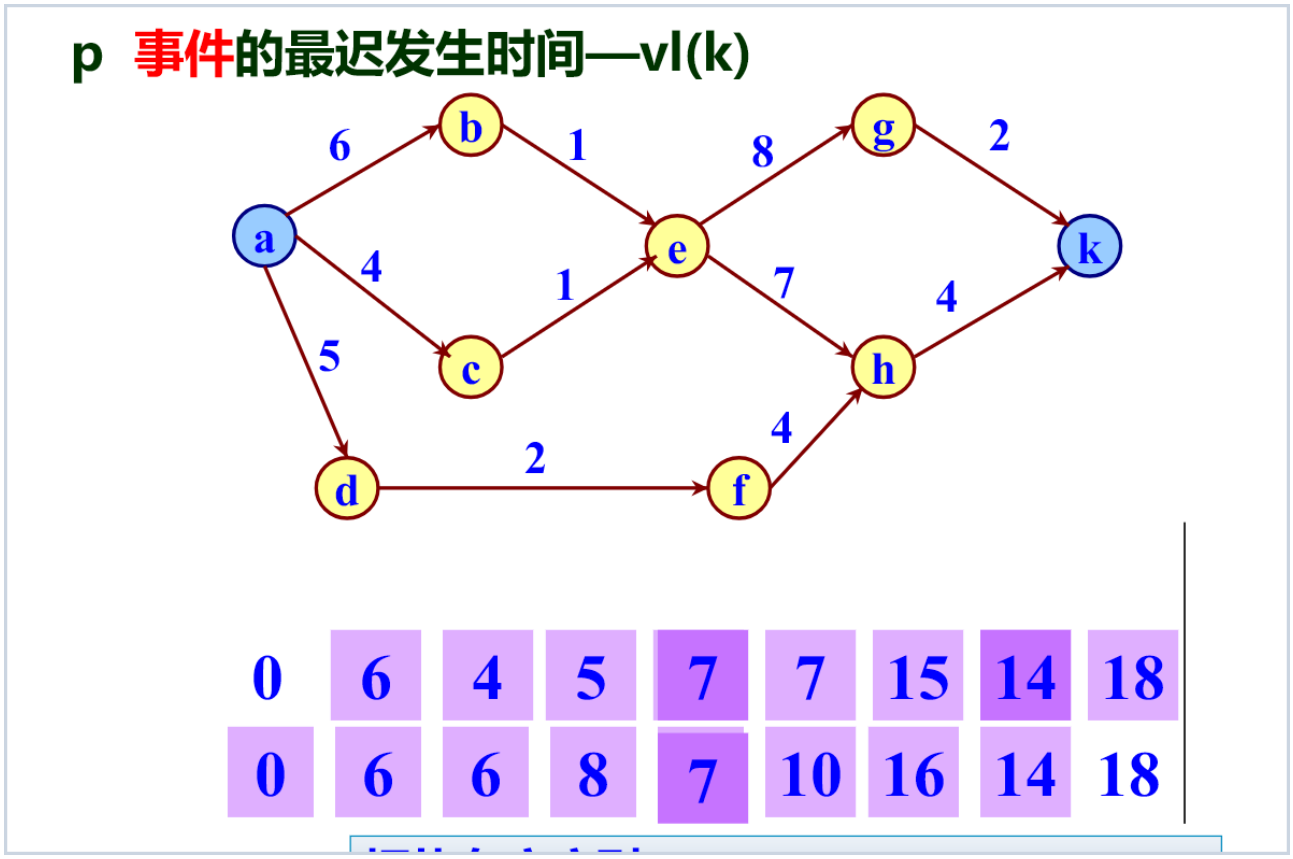


Figure 18: vl 反向计算

$$vl[u] = \min_{(u,v)\in E}\{vl[v] - w(u,v)\}$$

5.2.4 活动最早与最迟开始时间

对于活动 $a_i = (u, v)$：

$$e(i) = ve[u]$$

$$l(i) = vl[v] - w(u,v)$$

时差：

$$\text{slack}(i) = l(i) - e(i)$$

关键活动满足：

$$\boxed{e(i) = l(i)}$$

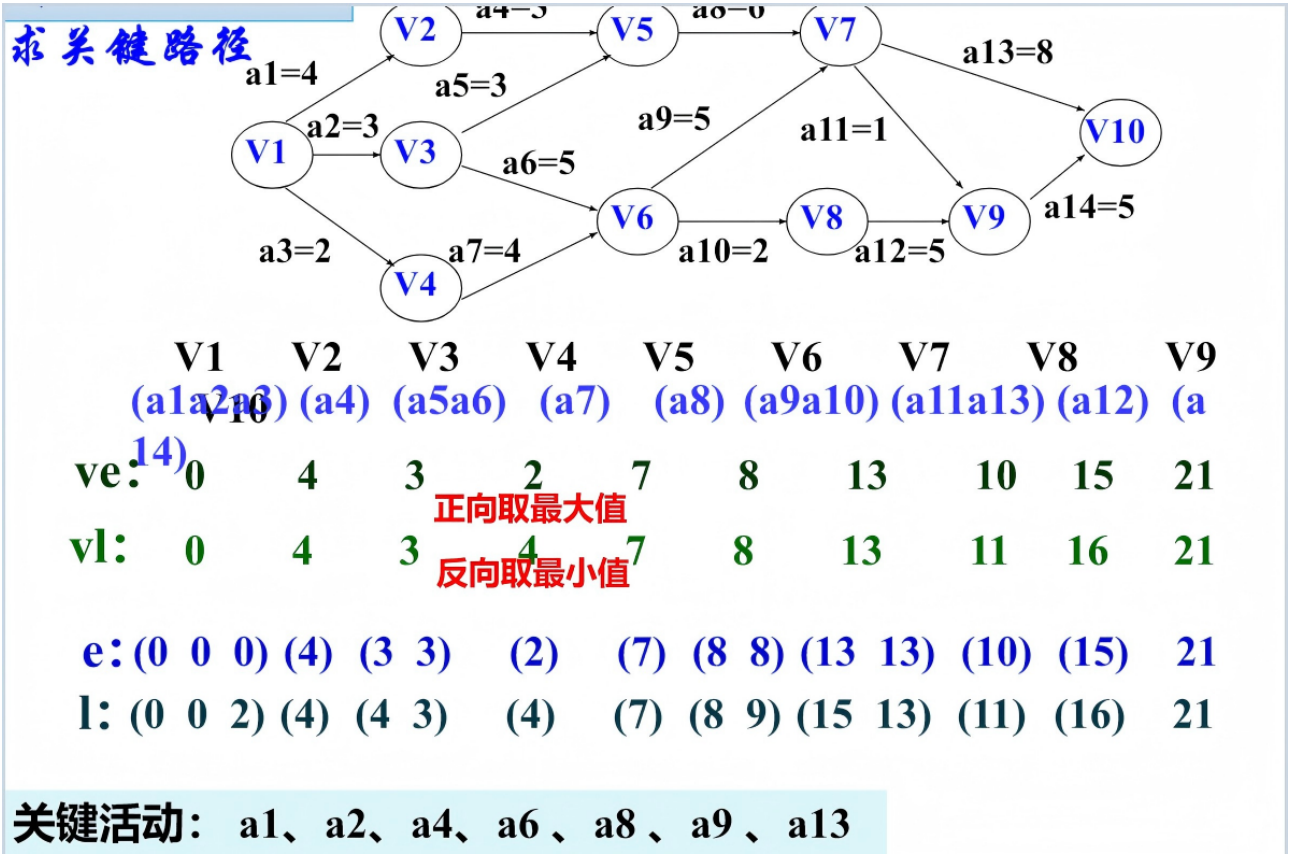


Figure 19: 关键路径完整计算

图 5-5　ve、vl、e、l 与关键活动

5.3 【典型题型精析】

题型 1：补全基于栈的拓扑排序（2025 秋往年题）

题干呈现 2025 秋手写代码题给出顶点 $0, 1, 2, 3, 4$ 和边

`0→1, 0→2, 1→3, 2→3, 3→4`

要求使用栈得到拓扑序列并补全邻接表版本代码。

解题思路分析 核心是维护入度：

- 初始化时把全部入度为 0 的顶点压栈；
- 删除出边时执行前置递减；
- 当入度恰好减到 0 时压栈。

详细解答步骤

```
bool TopologicalSort(Graph G) {
    stack<int> st;
    int count = 0;

    for (int v = 0; v < V; ++v) {
        if (indegree[v] == 0) {
            st.push(v);
        }
    }

    while (!st.empty()) {
        int u = st.top();
        st.pop();
        printf("%d ", u);
        ++count;

        for (int v : Adj[u]) {
            if (--indegree[v] == 0) {
                st.push(v);
            }
        }
    }

    return count == V;
}
```

若 `count < V`，剩余顶点均无法降到入度 0，说明剩余子图中存在有向环。

易错点与总结

- 判断必须写成 `--indegree[v] == 0`，避免重复入栈。
- 使用栈、队列或优先队列可能产生不同拓扑序。
- 拓扑序列可能不唯一，但“能否输出全部顶点”的判环结论唯一。

题型 2：AOE 网与所有关键路径（2025 春、2026 春往年题）

3. 给定下图所示的有向带权图，其中顶点表示事件，边表示活动，权重表示活动的

（1）写出该图的四个不同的拓扑排序序列。

（2）计算从源点 1 到其他各个顶点的最长路径长度。给出所有顶点的事件最早发

（3）找出该图的关键路径，并指出其长度。

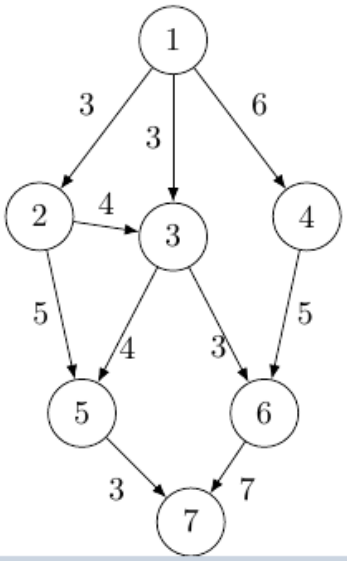


Figure 20: 2025 春 AOE 综合题

题干呈现

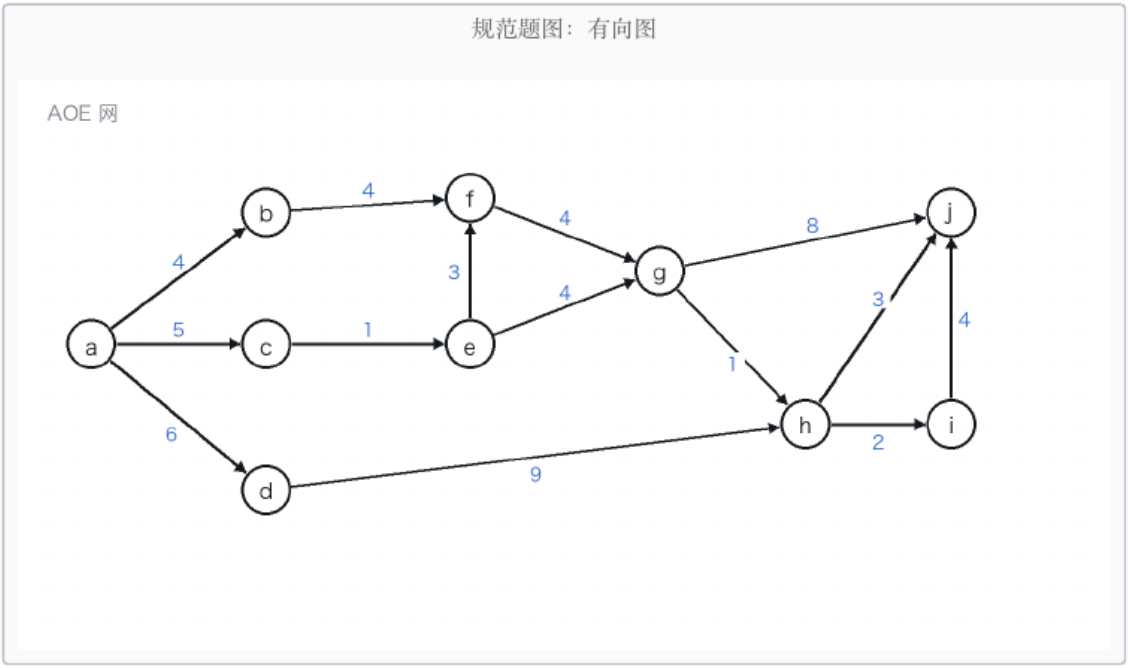
往年题 5-1　2025 春：四个拓扑序、ve 与关键路径

往年题 5-2　2026 春：邻接表、两个拓扑序与所有关键路径

两次考试均要求围绕同一张 AOE 网连续完成：

1. 写出邻接表或多个拓扑序列；
2. 计算所有事件的 ve；
3. 计算所有事件的 vl；
4. 求关键活动；
5. 列出全部关键路径。

5. 已知如图所示有向无环图，弧上的数字表示相应活动的持续时间。



1. 写出该图的邻接表。
2. 写出该图的两个不同的拓扑序列。
3. 求从顶点 a 到顶点 j 的最长路径，并写出所有的关键路径。

Figure 21: 2026 春 AOE 综合题

解题思路分析　必须依次完成：

拓扑排序
→ 正向计算 `ve`
→ 逆向计算 `vl`
→ 计算 `e` 与 `l`
→ 找 `l-e=0` 的活动
→ 沿关键活动从源点枚举到汇点

详细解答步骤　**第一步：2025 春题的拓扑序。**

四个合法示例为：

```
1,2,3,4,5,6,7
1,2,3,4,6,5,7
1,2,4,3,5,6,7
1,4,2,3,6,5,7
```

每个序列都满足：每条弧的起点出现在终点之前。

第二步：2025 春题正向计算 `ve`。

初始化 `ve[1]=0`，按拓扑序扫描每条边：

$$ve[v] = \max(ve[v], ve[u] + w(u,v))$$

得到：

顶点	1	2	3	4	5	6	7
ve	0	3	7	6	11	11	18

例如：

$$ve[3] = \max(0+3, 3+4) = 7$$

$$ve[7] = \max(11+3, 11+7) = 18$$

因此工程最短完成时间为 18。反向计算 `vl` 后，零时差活动组成：

$$\boxed{1 \to 4 \to 6 \to 7}$$

关键路径长度：

$$6+5+7=\boxed{18}$$

第三步：2026 春题列邻接表并计算最长距离。

按字母升序组织邻接表：

```
a: (b,4), (c,5), (d,6)
b: (f,4)
c: (e,1)
d: (h,9)
e: (f,3), (g,4)
f: (g,4)
g: (h,1), (j,8)
h: (i,2), (j,3)
i: (j,4)
j: ∅
```

两种合法拓扑序示例：

```
a,b,c,d,e,f,g,h,i,j
a,d,c,e,b,f,g,h,i,j
```

正向计算得到：

顶点	a	b	c	d	e	f	g	h	i	j
ve	0	4	5	6	6	9	13	15	17	21

汇点 j 的最大值由两个前驱分支最终同时取得，因此必须分别反向追踪。

所有关键路径为：

$$\boxed{a\to c\to e\to f\to g\to j}$$

长度：

$$5+1+3+4+8=21$$

以及：

$$\boxed{a\to d\to h\to i\to j}$$

长度：

$$6+9+2+4=21$$

若题目要求完整的 `vl` 与关键活动表，再令汇点：

$$vl[j]=ve[j]=21$$

并按逆拓扑序扫描：

$$vl[u]=\min(vl[u],vl[v]-w(u,v))$$

最后对每条活动边 (u,v) 计算：

$$e=ve[u],\qquad l=vl[v]-w(u,v)$$

选择 $e=l$ 的活动，再从源点沿关键活动做 DFS，直到汇点。

易错点与总结

- `ve` 正向取最大值，`vl` 反向取最小值。
- 关键活动可能组成多条关键路径。
- 不能只看某条“最长路径”的外观，必须通过 $e=l$ 判定关键活动。
- 工程最短完成时间等于汇点的 `ve`。

5.4 【自查与巩固建议】

自查提问

1. 为什么 DAG 一定存在入度为 0 的顶点?
2. 为什么“存在一个入度为 0 的顶点”不能推出图一定是 DAG?
3. 拓扑序不唯一的原因是什么?
4. 为什么 `ve` 取最大值而 `vl` 取最小值?
5. 如何列出全部关键路径，而不是只列一条?

关键词填空

1. 拓扑排序成功输出全部顶点的必要前提是图为 _____。
2. AOE 网中工程最短完成时间等于汇点的 _____。
3. 活动 (u,v) 的最迟开始时间为 _____。
4. 关键活动满足 _____。

参考答案：DAG；`ve`；$vl[v]-w(u,v)$；$e=l$。

进一步练习方向

- 对同一个 DAG 分别使用栈与队列生成拓扑序。
- 手写入度变化表。
- 对一个含分叉和汇合的 AOE 网枚举全部关键路径。

---

6. 最大流

6.1 【考点体系梳理】

知识逻辑如下：

```
容量网络
├─ 容量 c(u,v)
├─ 流量 f(u,v)
├─ 容量限制
├─ 流量守恒
└─ 最大流
   ├─ 残量网络
   ├─ 增广路
   ├─ Ford-Fulkerson
```

本节最关键的思想是：

> 反向残量边允许算法撤销一部分旧流量，从而修正早期不理想的选择。

6.2 【核心概念与公式】

6.2.1 容量限制

对每条边：

$$0 \le f(u,v) \le c(u,v)$$

6.2.2 流量守恒

对除源点 s 和汇点 t 外的任意顶点 v：

$$\sum_{u} f(u,v) = \sum_{w} f(v,w)$$

6.2.3 流值

$$|f| = \sum_v f(s,v) - \sum_v f(v,s)$$

6.2.4 残量网络

原边 (u,v) 容量为 c、当前流量为 f：

- 正向残量容量：

$$c_f(u,v) = c(u,v) - f(u,v)$$

- 反向残量容量：

$$c_f(v,u) = f(u,v)$$

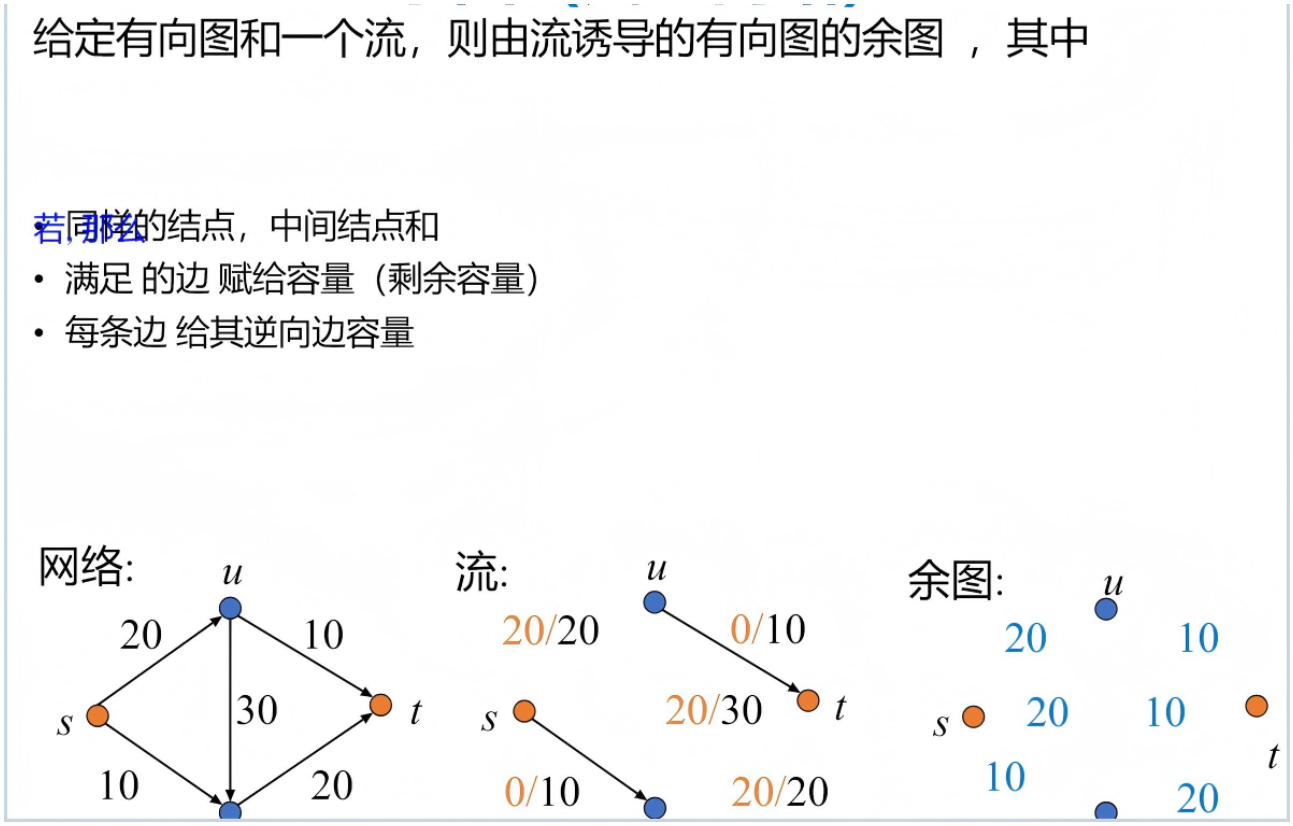


Figure 22: 正向残量

图 6-1　正向剩余容量与反向残量边

6.2.5 增广路与瓶颈

若残量网络中存在从 s 到 t 的路径 p，其瓶颈容量为：

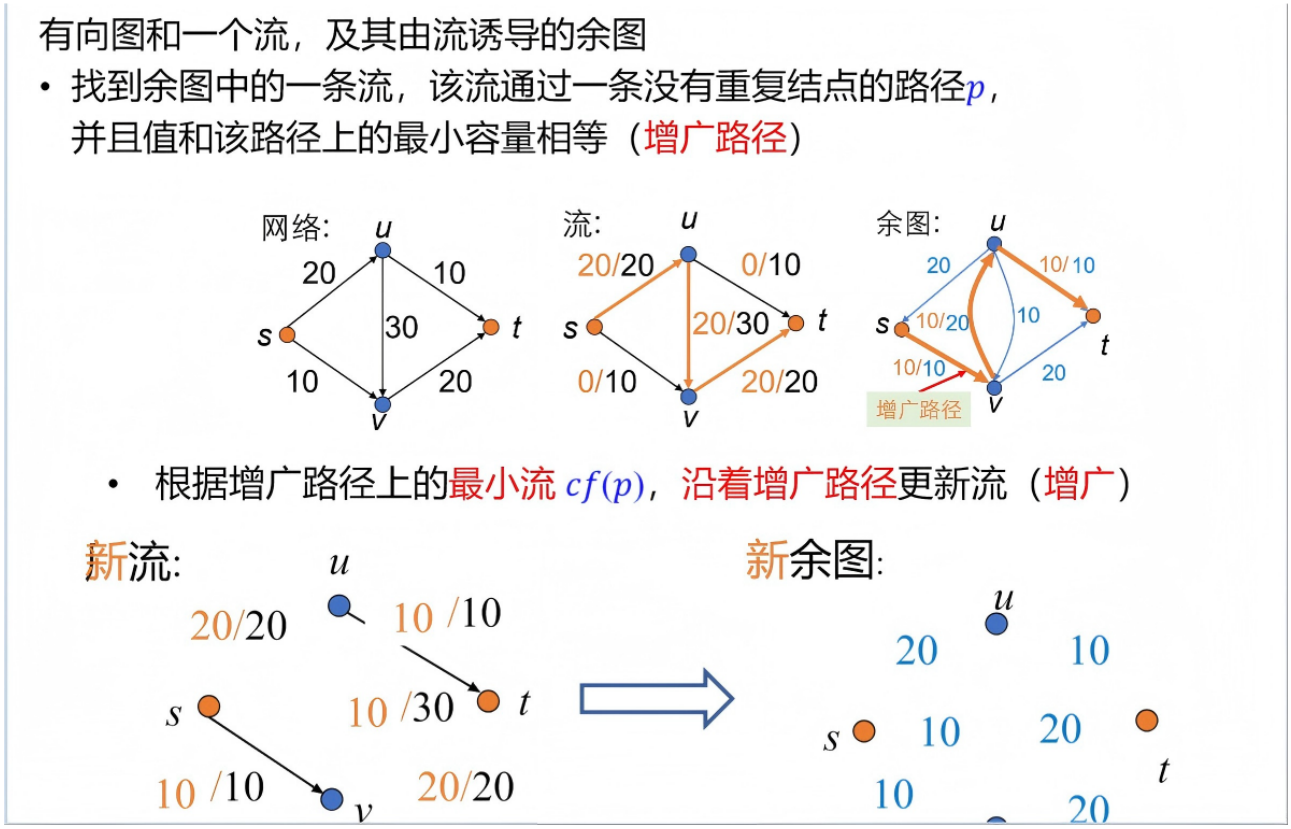


Figure 23: 反向残量

$$c_f(p) = \min_{(u,v)\in p} c_f(u,v)$$

沿路径增加该瓶颈流量，并同步更新正反向残量边。

图 6-2　Ford-Fulkerson 的增广与残量更新

6.3 【典型题型精析】

题型 1：最大流手算与最小割验证（2026 春往年题）

题干呈现

往年题 6-1　2026 春：借助流图与余图求汇点 t 的最大接受流量

题图容量为：

```
s→v1:6, s→v3:8, v3→v1:4,
v1→v2:10, v2→v3:2, v1→v4:3,
v3→v4:6, v2→t:4, v4→t:10
```

解题思路分析　每轮必须明确记录：

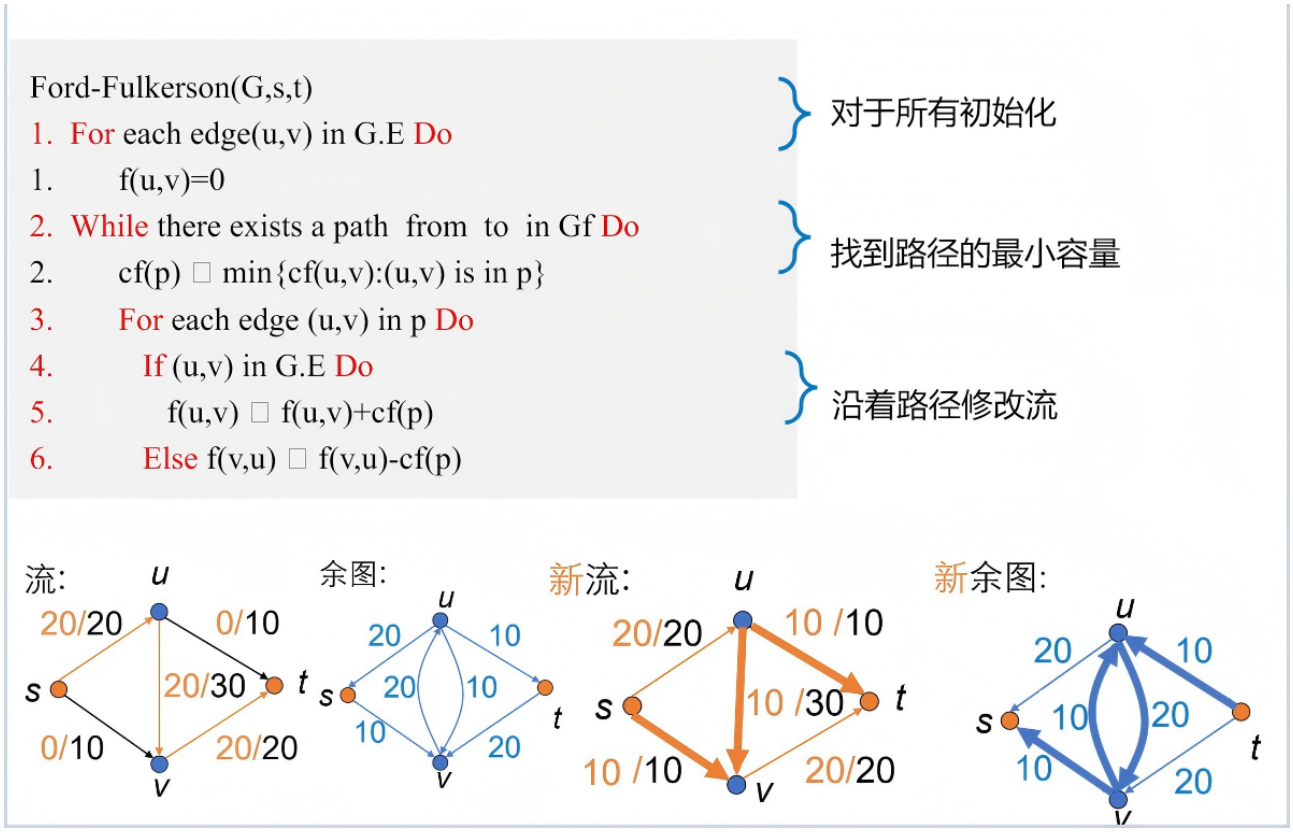


Figure 24: Ford-Fulkerson

当前残量网络

→ 选择增广路

→ 找路径瓶颈

→ 更新正向和反向残量

→ 累加当前流值

详细解答步骤

1. 初始化所有边流量为 0，在残量网络中寻找 s 到 t 的增广路。
2. 重复增广，直到残量网络中不存在 s 到 t 的增广路为止——这就是 Ford-Fulkerson 的终止条件。整数容量下每次增广至少增加 1 单位流量，必然终止；一般无理容量下若路径选择不当可能不终止。实际实现常用 BFS 找最短增广路（Edmonds-Karp），保证多项式时间。
3. 每条路径的增广量为：

$$\Delta = \min_{(u,v)\in p} c_f(u,v)$$

3. 一组可行的最终流如下：

边	流量/容量
$s \to v_1$	$6/6$
$s \to v_3$	$7/8$

边	流量/容量
$v_3 \to v_1$	$1/4$
$v_1 \to v_2$	$4/10$
$v_1 \to v_4$	$3/3$
$v_3 \to v_4$	$6/6$
$v_2 \to t$	$4/4$
$v_4 \to t$	$9/10$

检查中间顶点流量守恒：

v1：流入 6+1=7，流出 4+3=7
v2：流入 4，流出 4
v3：流入 7，流出 1+6=7
v4：流入 3+6=9，流出 9

因此得到可行流：

$$|f| = 6 + 7 = \boxed{13}$$

4. 使用最小割验证。取：

$$S = \{s, v_1, v_2, v_3\}, \qquad T = \{v_4, t\}$$

从 S 指向 T 的割边为：

v1→v4，容量 3
v3→v4，容量 6
v2→t，容量 4

割容量：

$$c(S,T) = 3 + 6 + 4 = \boxed{13}$$

已经找到流值为 13 的可行流，同时找到容量为 13 的割。根据最大流最小割定理：

$$\boxed{\text{最大流} = 13}$$

5. 在实际增广过程中，对正向原边执行：

$$f(u,v) \leftarrow f(u,v) + \Delta$$

若路径使用反向残量边，则撤回对应原边流量：

2. 已知如图所示带容量的有向网络，弧旁数字表示相应弧的容量。求汇点 t 的最大接受流
提示：可借助流图和余图进行求解。

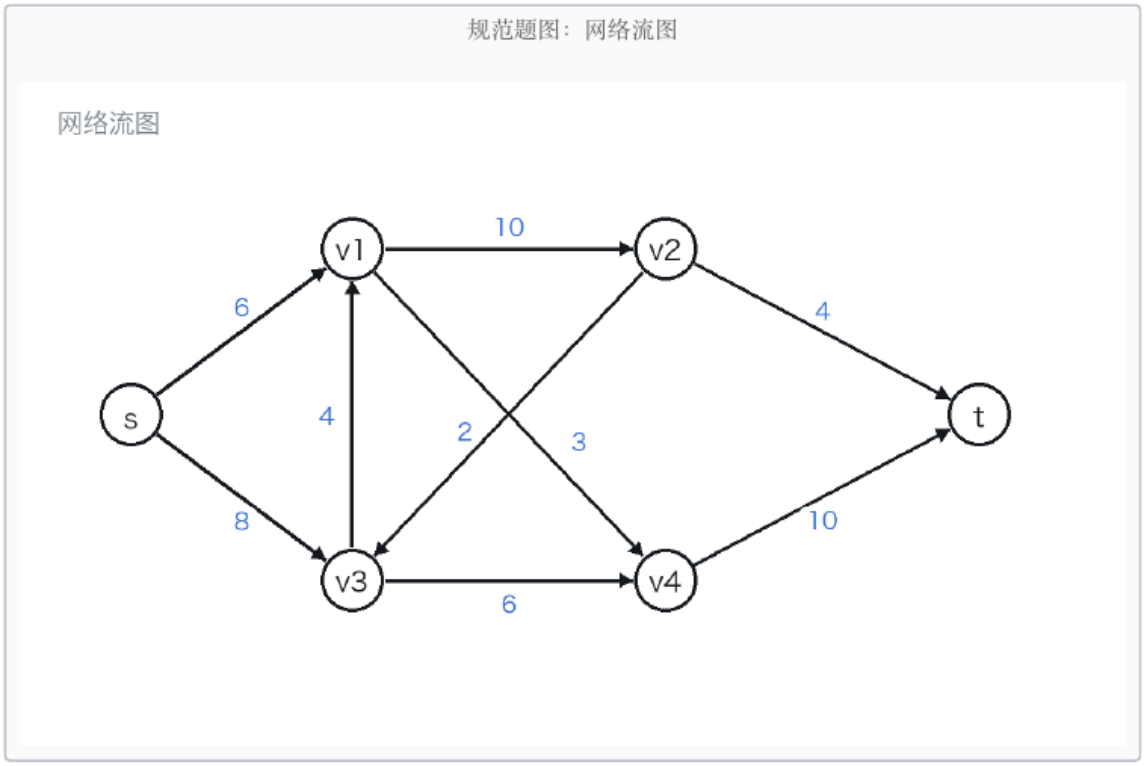


Figure 25: 2026 春最大流题

$$f(v,u) \leftarrow f(v,u) - \Delta$$

重复到残量网络中不存在 s 到 t 的路径。

易错点与总结

- 增广后必须同时更新正向与反向残量容量。
- 瓶颈取路径上的最小残量，不是最小原始容量。
- 只比较源点总出容量和汇点总入容量只能得到上界，不能证明上界一定能实现。
- 最大流值等于最小割容量。

6.4 【自查与巩固建议】

自查提问

1. 反向残量边的容量为什么等于当前流量?
2. 一条边饱和后，其正向和反向残量分别是多少?
3. 增广路的瓶颈如何计算?
4. 最大流结束时如何构造一个最小割?

构造方法：在最终的残量网络中，从源点 s 出发沿残量容量大于 0 的边做遍历，可达的顶点构成集合 S，其余顶点构成 T，则 (S,T) 就是最小割，其容量恰好等于最大流值。5. 为什么只计算源点总出容量不能直接断定最大流?

关键词填空

1. 增广量等于路径上的最小 _____。
2. 原边当前流量为 f 时，反向残量容量为 _____。
3. Edmonds-Karp 使用 _____ 寻找增广路。
4. 最大流值等于最小 _____ 容量。

参考答案：残量容量；f；BFS；割。

进一步练习方向

- 对一个四层容量网络画出每轮残量图。
- 有意选择一条不理想的首轮增广路，观察反向边如何修正。
- 为 2026 春题写出至少两组不同的合法增广路顺序，并验证最终流值不变。

---

7. 全章自测与考前检查

7.1 【考点体系梳理】

全章算法选择链：

先判断图的类型
├─ 无向/有向
├─ 无权/带权
├─ 是否存在负边
├─ 是否为 DAG
└─ 单源/多源/所有点对

再确定目标
├─ 遍历与连通性：DFS/BFS
├─ 无权最短路：BFS
├─ 最小连接代价：Prim/Kruskal
├─ 非负权单源：Dijkstra
├─ 含负边单源：Bellman-Ford
├─ 所有点对：Floyd
├─ 先修关系与判环：拓扑排序
├─ 工程工期：关键路径
└─ 最大输送能力：最大流

7.2 【核心概念与公式】

算法	主要适用条件	邻接表典型复杂度	核心状态
DFS/BFS	一般图	$O(V+E)$	`visited`、栈/队列
Prim	连通无向带权图	$O(E\log V)$	`lowcost`
Kruskal	连通无向带权图	$O(E\log E)$	排序边、并查集
Dijkstra	非负权单源	$O(E\log V)$	`dist`、已定型集合
Bellman-Ford	可含负边	$O(VE)$	全部边的多轮松弛
Floyd	所有点对	$O(V^3)$	距离矩阵、中间点矩阵
拓扑排序	DAG/AOV	$O(V+E)$	`indegree`
关键路径	AOE 网	$O(V+E)$	`ve`、`vl`、`e`、`l`

7.3 【典型题型精析】

题型：综合算法选择

题干呈现　为下列问题选择首选算法，并说明理由：

1. 社交网络中求两人之间最少经过多少次转发。

2. 带负权边但无负环的单源最短路径。
3. 课程先修关系是否存在循环依赖。
4. 城市之间铺设最低成本道路。
5. 求任意两个城市之间的最短距离。
6. 项目最短工期与不能延误的活动。
7. 容量网络中汇点能够接收的最大流量。

解题思路分析 先识别图的性质，再识别输出目标，最后选择算法。不能只看到“最短”二字就使用 Dijkstra。

详细解答步骤

场景	首选算法	选择理由
最少转发次数	BFS	无权最短路
含负边单源最短路	Bellman-Ford	允许负边
先修循环依赖	拓扑排序	输出数小于 V 即有环
最低铺路成本	Prim/Kruskal	最小生成树
任意点对最短路	Floyd	所有点对
项目工期	关键路径	AOE 网
网络最大输送量	Ford-Fulkerson/Edmonds-Karp	容量网络与残量图

易错点与总结

- “最短路径”不是一个单一算法。
- 算法名称之后必须补充适用条件。
- 过程题应展示状态变化，不能只给最终结果。

7.4 【自查与巩固建议】

高频判断

1. 无向图邻接矩阵一定对称。
2. 有向图邻接矩阵一定不对称。
3. 邻接表下 DFS 与 BFS 均为 $O(V+E)$。
4. 有向图只要存在一个入度为 0 的顶点，就一定是 DAG。
5. Dijkstra 可以处理负边，只要不存在负环。
6. AOE 网中的关键路径是源点到汇点的最长路径。
7. 关键活动一定只组成一条关键路径。
8. Ford-Fulkerson 的反向残量边允许撤回已有流量。
9. 最小生成树一定唯一。

参考答案：

1 对
2 错
3 对
4 错
5 错
6 对
7 错
8 对
9 错

手写算法检查表

- ☐ 初始化数组是否完整?
- ☐ BFS 是否在入队时标记?
- ☐ Dijkstra 是否只在非负权图上使用?
- ☐ Bellman-Ford 是否检查 `dist[u] != INF`?
- ☐ Floyd 是否把中间点 k 放在最外层?
- ☐ 拓扑排序是否统计最终输出顶点数?
- ☐ 关键路径是否按“`ve` 最大、`vl` 最小”计算?
- ☐ 最大流是否更新了正向和反向残量?
- ☐ 是否写出时间复杂度与适用条件?
- ☐ 是否声明邻接点访问次序?

考前最后复盘

1. 看到图，先判断方向、权值、连通性和是否可能有环。
2. 看到算法题，先写适用条件，再写状态变量。
3. 看到过程题，使用表格记录每轮变化。
4. 看到遍历序列，先声明邻接点访问次序。
5. 看到关键路径，必须考虑是否存在多条。
6. 看到最大流，必须画或维护残量网络。